第5章　数组和稀疏矩阵

提纲

CONTENTS

5.1　数组

5.2　特殊矩阵的压缩存储

5.3　稀疏矩阵

5.1 数 组

5.1.1 数组的概念

- 数组是二元组（idx，value）的集合，对每个idx，都有一个value值与之对应。idx称为下标，可以由一个整数、两个整数或多个整数构成，下标含有d（$d \geq 1$）个整数称为维数是d。
- 数组按维数分为一维、二维和多维数组。
- 一维数组A是n（$n>1$）个相同类型元素a_0，a_1，…，a_{n-1}构成的有限序列，其逻辑表示为$A=(a_0, a_1, \cdots, a_{n-1})$，其中，$A$是数组名，$a_i$（$0 \leqslant i \leqslant n-1$）是数组$A$中序号为$i$的元素。
- 一个二维数组可以看作是每个数据元素都是相同类型的一维数组的一维数组。
- 以此类推。

二维数组的逻辑关系用二元组表示

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{bmatrix}$$

B=(*D*, *R*)

$R=\{r_1, r_2\}$

r_1={<1, 2>, <2, 3>, <3, 4>, <5, 6>, <6, 7>, <7, 8>, <9, 10>,
<10, 11>, <11, 12>} //同行关系

r_2={<1, 5>, <5, 9>, <2, 6>, <6, 10>, <3, 7>, <7, 11>, <4, 8>,
<8, 12>} //同列关系

数组具有以下特点

（1）数组中各元素都具有统一的数据类型。

（2）d（$d \geq 1$）维数组中的非边界元素具有d个前驱元素和d个后继元素。

（3）数组维数确定后，数据元素个数和元素之间的关系不再发生改变，特别适合于顺序存储。

（4）每个有意义的下标都存在一个与其相对应的数组元素值。

d维数组抽象数据类型

ADT Array

{

数据对象：

D={ 数组中所有元素 }

数据关系：

R={r_1, r_2, …, r_d}

r_i={ 元素之间第i维的线性关系 | i=1, …, d}

基本运算：

Value(A, i_1, i_2, …, i_d)：A是已存在的d维数组，其运算结果是返回$A[i_1, i_2, \cdots, i_d]$值。

Assign(A, e, i_1, i_2, …, i_d)：A是已存在的d维数组，其运算结果是置$A[i_1, i_2, \cdots, i_d]=e$。

…

}

5.1.2 数组的存储结构

数组的主要操作是存取元素值，没有插入和删除操作，所以数组通常采用顺序存储方式来实现。

1．一维数组

- 一维数组的所有元素依逻辑次序存放在一片连续的内存存储单元中。
- 其起始地址为第一个元素a_0的地址即LOC(a_0)。
- 假设每个数据元素占用k个存储单元。
- 则任一数据元素a_i的存储地址LOC(a_i)就可由以下公式求出

$$LOC(a_i)=LOC(a_0)+i\times k \qquad (1\leqslant i<n)$$

一维数组具有随机存储特性

- C++中一维数组的定义方式：T a[M];其中M为常量，T为数组元素类型。
- 也可以使用vector<T>容器作为一维动态数组。

2．d维数组

以m行n列的二维数组$A_{m\times n}=(a_{i,j})$为例讨论（二维数组也称为矩阵）。

$$\begin{pmatrix} a_{0,0} & a_{0,1} & \cdots & a_{0,n-1} \\ a_{1,0} & a_{1,1} & \cdots & a_{1,n-1} \\ \cdots & & & \\ a_{m-1,0} & a_{m-1,1} & \cdots & a_{m-1,n-1} \end{pmatrix}$$

$$\begin{pmatrix} a_{0,0} & a_{0,1} & \cdots & a_{0,n-1} \\ a_{1,0} & a_{1,1} & \cdots & a_{1,n-1} \\ \cdots & & & \\ a_{m-1,0} & a_{m-1,1} & \cdots & a_{m-1,n-1} \end{pmatrix}$$

按行优先存储

假设每个元素占k个存储单元，$LOC(a_{0,0})$表示$a_{0,0}$元素的存储地址。对于元素$a_{i,j}$：

- $a_{i,j}$前面有$0 \sim i-1$共i行，每行n个元素，共有$i \times n$个元素。
- 在第i行中前面有$a[i,0..j-1]$，共j个元素。
- 合起来，$a_{i,j}$前面有$i \times n+j$个元素。

$$LOC(a_{i,j})=LOC(a_{0,0}) + (i \times n + j) \times k$$

$$\begin{pmatrix} a_{0,0} & a_{0,1} & \cdots & a_{0,n-1} \\ a_{1,0} & a_{1,1} & \cdots & a_{1,n-1} \\ \cdots & & & \\ a_{m-1,0} & a_{m-1,1} & \cdots & a_{m-1,n-1} \end{pmatrix}$$

按列优先存储

假设每个元素占k个存储单元，$LOC(a_{0,0})$表示$a_{0,0}$元素的存储地址。对于元素$a_{i,j}$：

- $a_{i,j}$前面有$0 \sim j-1$共j列，每列m个元素，共有$j \times m$个元素。
- 在第j列中前面有$a[0..i-1,j]$，共i个元素。
- 合起来，$a_{i,j}$前面有$j \times m+i$个元素。则：

$$LOC(a_{i,j})=LOC(a_{0,0}) + (j \times m + i) \times k$$

二维数组也具有随机存储特性，以此类推。

更一般地，数组$A[c_1..d_1, c_2..d_2]$，则该数组按行优先存储时有：

$$LOC(a_{i,j})=LOC(a_{c1,c2})+[(i-c_1)\times(d_2-c_2+1)+(j-c_2)]\times k$$

按列优先存储时有：

$$LOC(a_{i,j})=LOC(a_{c1,c2})+[(j-c_2)\times(d_1-c_1+1)+(i-c_1)]\times k$$

- C++中二维数组的定义方式：T a[M][N];其中M、N为常量，T为数组元素类型。
- 也可以使用vector<vector<T>>容器作为二维动态数组。

【例5.1】设有二维数组a[1..50，1..80]，其a[1][1]元素的地址为2000，每个元素占2个存储单元，若按行优先存储，则元素a[45][68]的存储地址为多少?若按列优先存储，则元素a[45][68]的存储地址为多少？

按行优先存储

- 元素a[45][68]前面有1～44行，每行80个元素，计44×80个元素。
- 在第45行中，元素a[45][68]前面有a[45][1..67]计67个元素，这样元素a[45][68]前面存储的元素个数=44×80+67。
- LOC(a[45][68])=2000+(44×80+67)×2=9174。

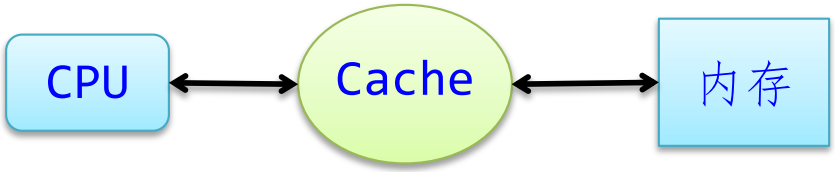
【例5.1】设有二维数组a[1..50，1..80]，其a[1][1]元素的地址为2000，每个元素占2个存储单元，若按行优先存储，则元素a[45][68]的存储地址为多少?若按列优先存储，则元素a[45][68]的存储地址为多少？

按列优先存储

- 元素a[45][68]前面有1～67列，每列50个元素，计67×50个元素。
- 在第68列中，元素a[45][68]前面有a[1..44][68]计44个元素，这样元素a[45][68]前面存储的元素个数=67×50+44。
- LOC(a[45][68])=2000+(67×50+44)×2=8788。

程序局部性原理

- 在C++语言中二维及以上维的数组就是按行优先存储的。
- 在程序中采用数组存放大量的数据时，这些数据存放在内存中，当CPU读数组中的元素时并不是立即访问内存，而是先访问Cache（高速缓存，其速度比访问内存快得多）。
- 如果访问的数据在Cache中便直接取相应的数据（称为命中），如果访问的数据不在Cache中才访问内存，并将访问数据所在的一个页块调入Cache。



```
//程序A
int a[1000][50][8000];
int main()
{  for (int i=0;i<1000;i++)
     for (int j=0;j<50;j++)
        for (int k=0;k<8000;k++)
           a[i][j][k]=i+j+k;
   return 0;
}
```

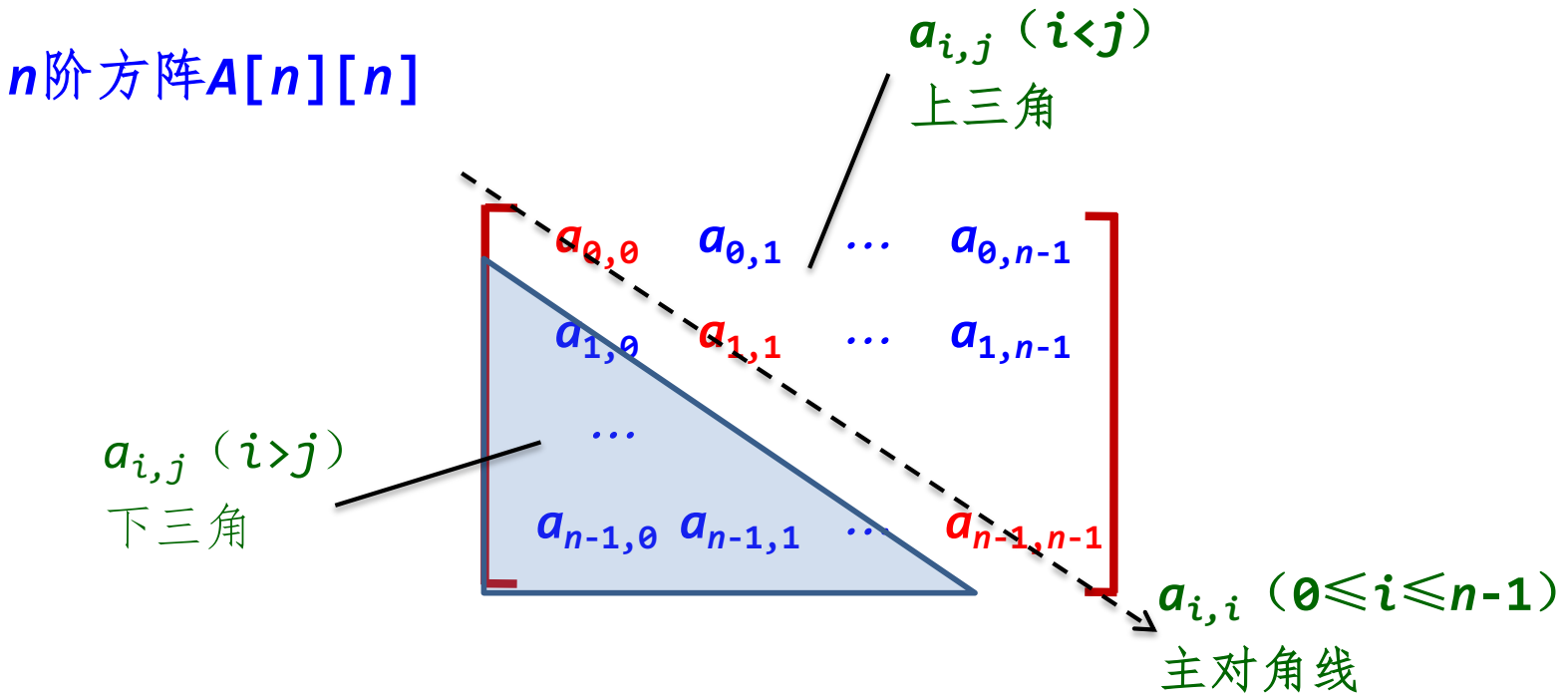
```
//程序B
int a[1000][50][8000];
int main()
{  for (int k=0;k<8000;k++)
     for (int j=0;j<50;j++)
        for (int i=0;i<1000;i++)
           a[i][j][k]=i+j+k;
   return 0;
}
```

程序A的执行时间为1.597秒，而程序B的执行时间为17.83秒，相差10多倍。？

5.1.3 数组的应用

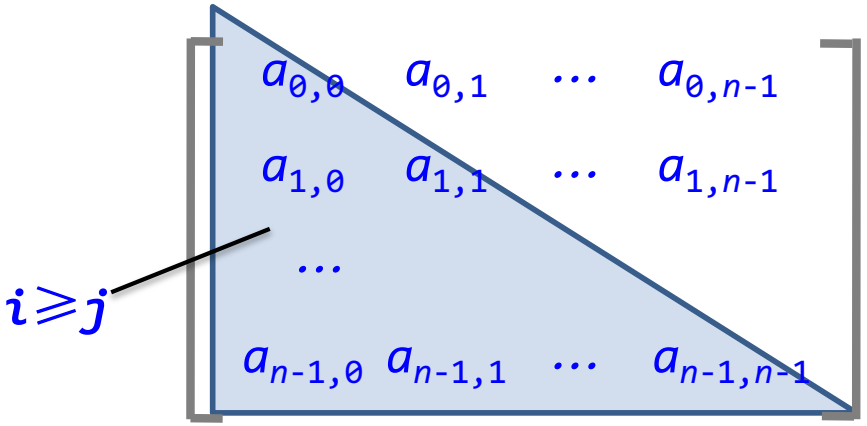
- 由于数组使用简单方便，特别是具有随机存取特性，因此在编程中数组被广泛地使用。
- 使用数组目的一方面为了储存大量的数据类型相同的数据，避免重复性操作，另一方面用于模拟现实世界，例如顺序表就是采用数组模拟线性表。

5.2 特殊矩阵的压缩存储



5.2.1 对称矩阵的压缩存储

若一个n阶方阵A的元素满足$a_{i,j}=a_{j,i}$（$0 \leqslant i, j \leqslant n-1$），则称其为$n$阶对称矩阵。



下三角+主对角线

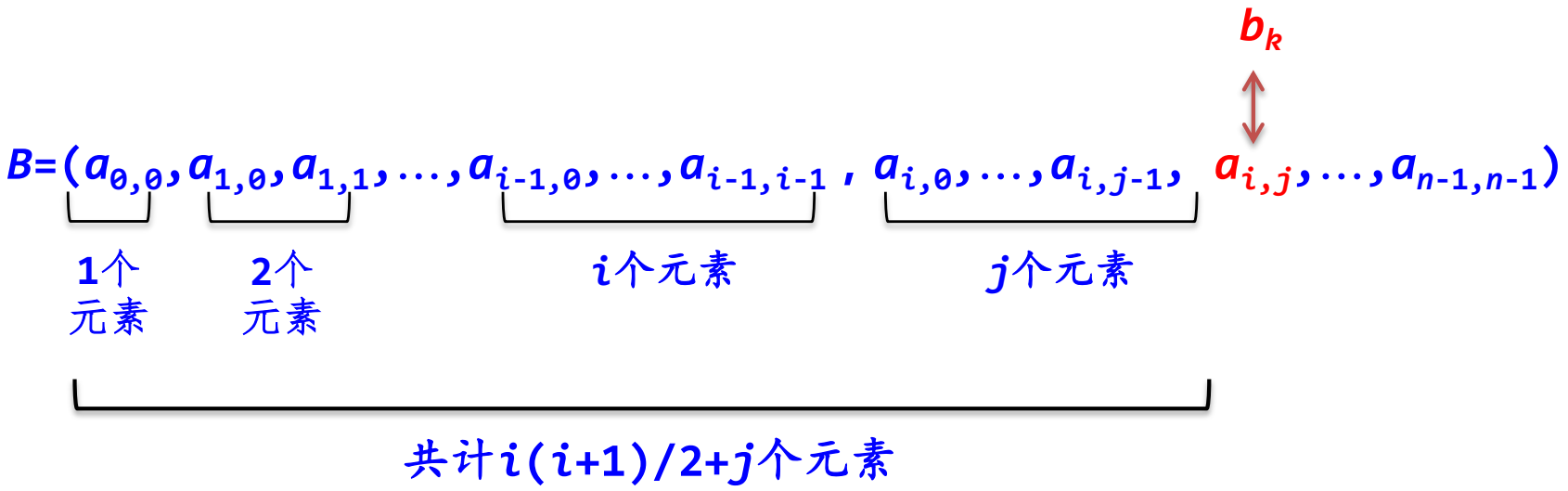
$a_{i,j}$ → k = ?

b_k

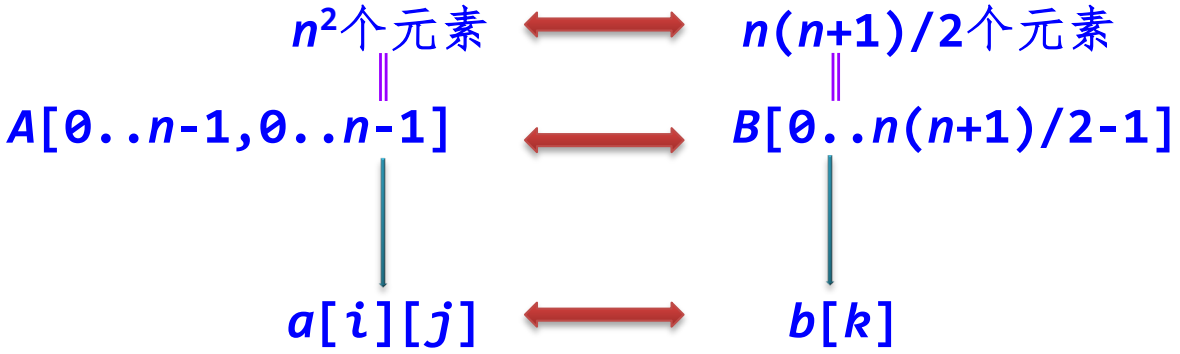
$a_{0,0}$，$a_{1,0}$，$a_{1,1}$，…，$a_{n-1,0}$，$a_{n-1,1}$，…，$a_{n-1,n-1}$

$B=(b_0, b_1, b_2, \ldots\ldots, \ldots\ldots, b_s)$

$n(n+1)/2$个元素



$$k=\begin{cases}\dfrac{i(i+1)}{2}+j & \text{当}i\geqslant j\text{时（下三角+主对角线的元素）}\\ \dfrac{j(j+1)}{2}+i & \text{当}i<j\text{时（}a_{i,j}=a_{j,i}\text{）}\end{cases}$$

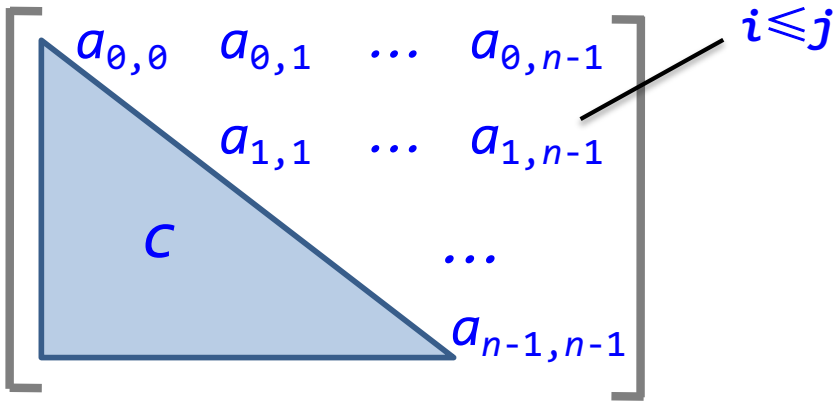


$$k=\begin{cases}\dfrac{i(i+1)}{2}+j & \text{当}i\geqslant j\text{时（下三角+主对角线的元素）}\\[2ex] \dfrac{j(j+1)}{2}+i & \text{当}i<j\text{时（}a_{i,j}=a_{j,i}\text{）}\end{cases}$$

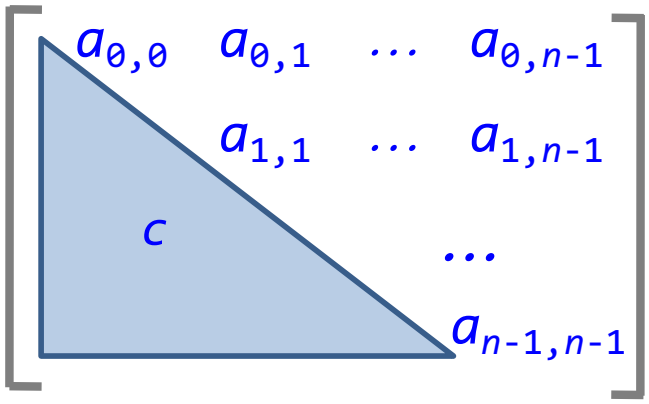
对于对称矩阵A，采用一维数组B存储，并提供A的所有运算。

5.2.2 三角矩阵的压缩存储

上三角矩阵



对于上三角部分的元素$a_{i,j}$



第0行：n个元素
第1行：n-1个元素
…
第i-1行：n-i+1个元素

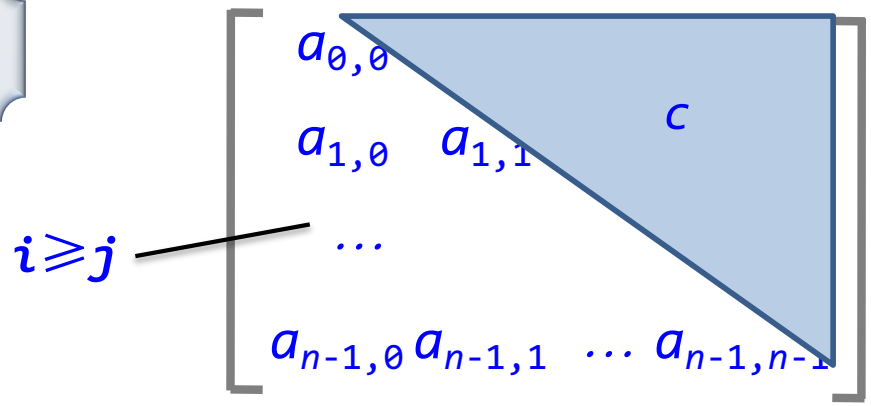
} i(2n-i+1)/2个元素

第i行有a[i,i..j-1]：j-i个元素

$$k=\begin{cases}\dfrac{i(2n-i+1)}{2}+j-i & \text{当 } i\le j \text{ 时}\\[2ex] \dfrac{n(n+1)}{2} & \text{当 } i>j \text{ 时}\end{cases}$$

存放常量c

下三角矩阵



$$k=\begin{cases}\dfrac{i(i+1)}{2}+j & \text{当 } i \geqslant j \text{ 时} \\ \dfrac{n(n+1)}{2} & \text{当 } i<j \text{ 时}\end{cases}$$

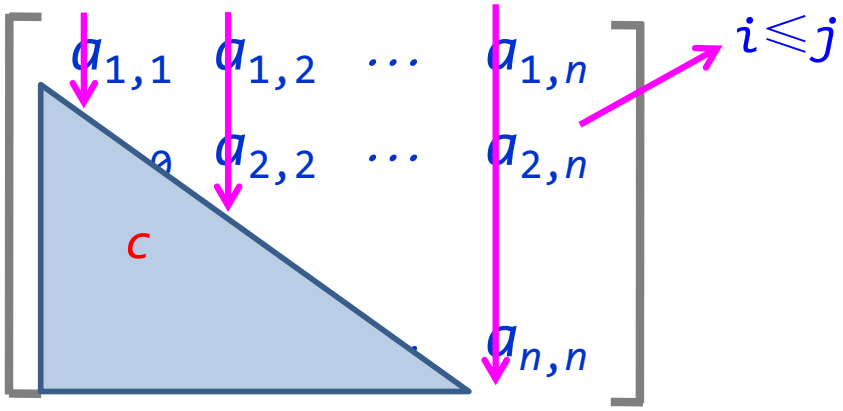
存放一个常量c

示例

若将n阶上三角矩阵A按列优先顺序压缩存放在一维数组$B[1..n(n+1)/2]$中，A中第一个非零元素$a_{1,1}$存于B数组的b_1中，则应存放到b_k中的非零元素$a_{i,j}$（$i \le j$）的下标i、j与k的对应关系是（　）。

A. $i(i+1)/2+j$　　B. $i(i-1)/2+j$

C. $j(j+1)/2+i$　　D. $j(j-1)/2+i$



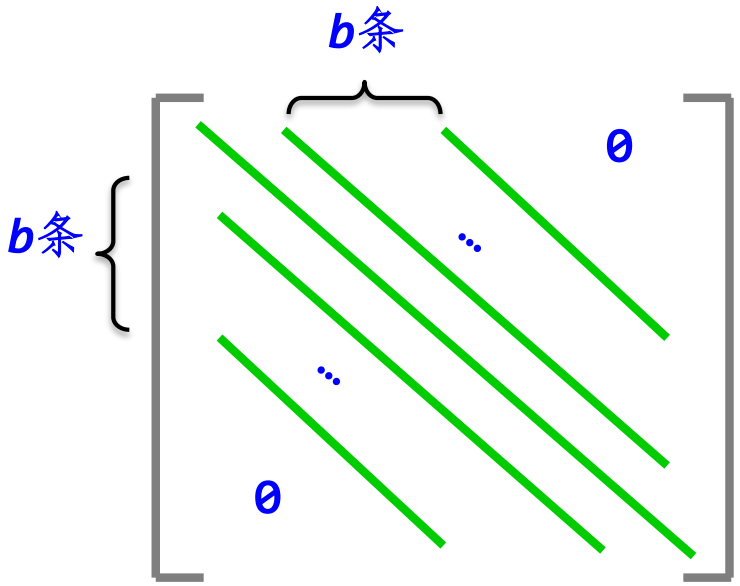
1～j-1列的元素个数：$j(j-1)/2$

第j列a_{ij}之前的元素个数：i-1

$k=j(j-1)/2+i-1+1=\ j(j-1)/2+i$

- 按行还是按列
- 初始下标从0还是从1开始

5.2.3 对角矩阵的压缩存储



半带宽为b的对角矩阵

对角矩阵　　压缩存储

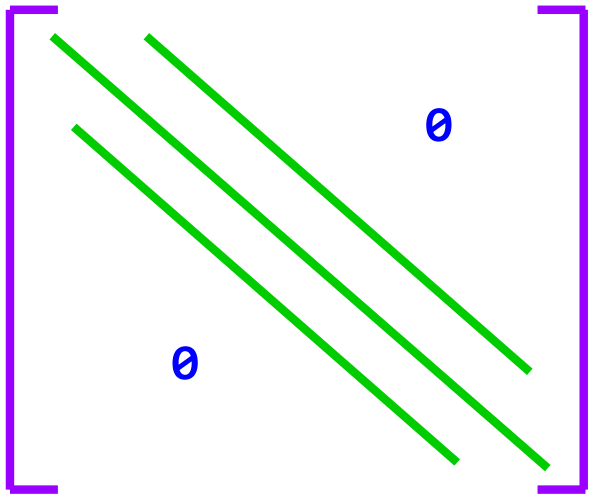
A ⟷ B

$a[i][j]$ ⟷ $b[k]$

当$b=1$时称为三对角矩阵

其压缩地址计算公式如下：

$$k = 2i + j$$



5.3 稀疏矩阵

一个阶数较大的矩阵中的非零元素个数s相对于矩阵元素的总个数t十分小时，即s<<t时，称该矩阵为**稀疏矩阵**。

例如一个100×100的矩阵，若其中只有100个非零元素，就可称其为稀疏矩阵。

定性的描述

稀疏矩阵和特殊矩阵的不同点：

- 特殊矩阵的特殊元素（值相同元素、常量元素）分布有规律。
- 稀疏矩阵的特殊元素（非0元素）分布没有规律。

5.3.1 稀疏矩阵的三元组表示

稀疏矩阵的压缩存储方法是只存储非零元素。

稀疏矩阵中的每一个非零元素需由一个三元组：

$$(i\ ,\ j\ ,\ a_{i,j})$$

唯一确定，稀疏矩阵中的所有非零元素构成三元组线性表。

5.3.1 稀疏矩阵的三元组表示

一个6×7阶稀疏矩阵A的三元组线性表表示

$$A_{6\times7}=\begin{bmatrix}0&0&1&0&0&0&0\\0&2&0&0&0&0&0\\3&0&0&0&0&0&0\\0&0&0&5&0&0&0\\0&0&0&0&6&0&0\\0&0&0&0&0&7&4\end{bmatrix}$$

i	j	$a_{i,j}$
0	2	1
1	1	2
2	0	3
3	3	5
4	4	6
5	5	7
5	6	4

通常按行优先顺序排列

把稀疏矩阵的三元组线性表按顺序存储结构存储，则称为稀疏矩阵的三元组顺序表。（和课本示例略有不同）

```
#define MaxSize  100   //矩阵中非零元素最多个数
typedef struct
{   int r;             //行号
    int c;             //列号
    ElemType d;        //元素值
} TupNode;             //三元组定义
typedef struct
{   int rows;          //行数值
    int cols;          //列数值
    int nums;          //非零元素个数
    TupNode data[MaxSize];
} TSMatrix;            //三元组顺序表定义
```

存放一个非0元素

存放整个稀疏矩阵

三元组表示中每个元素的类定义如下：

```
struct TupElem                          //单个三元组元素的类型
{  int r;                               //行号
   int c;                               //列号
   int d;                               //元素值
   TupElem() {}                         //构造函数
   TupElem(int r1,int c1,int d1)        //重载构造函数
   {  r=r1;
      c=c1;
      d=d1;
   }
};
```

设计稀疏矩阵三元组存储结构类TupClass如下：

```
class TupClass                          //三元组存储结构类
{   int rows;                           //行数
    int cols;                           //列数
    int nums;                           //非零元素个数
    TupElem* data;                      //稀疏矩阵对应的三元组顺序表
    //基本运算算法
};
```

TupClass类中包含如下基本运算方法：

- CreateTup(A,m,n)：由*m*行*n*列的稀疏矩阵*A*创建其三元组表示。
- Setvalue(i,j,x)：利用三元组给稀疏矩阵的元素赋值即执行*A*[*i*][*j*]=*x*。
- GetValue(i, j)：利用三元组取稀疏矩阵的元素值即执行*x*=*A*[*i*][*j*]。
- DispTup()：输出稀疏矩阵的三元组表示。

其中，data列表用于存放稀疏矩阵中所有非零元素，通常按行优先顺序排列。这种有序结构可简化大多数稀疏矩阵运算算法。

（1）从一个二维矩阵创建其三元组表示

以行序方式扫描二维矩阵A，将其非零的元素插入到三元组t的后面。

$$A_{6\times7}=\begin{bmatrix}0&0&1&0&0&0&0\\0&2&0&0&0&0&0\\3&0&0&0&0&0&0\\0&0&0&5&0&0&0\\0&0&0&0&6&0&0\\0&0&0&0&0&7&4\end{bmatrix}$$

t:

i	j	a_{ij}
0	2	1
1	1	2
2	0	3
3	3	5
4	4	6
5	5	7
5	6	4

约定：data域中表示的非零元素通常以行序为主序顺序排列，它是一种下标按行有序的存储结构。

这种有序存储结构可简化大多数矩阵运算算法。

```
void CreatMat(TSMatrix &t,ElemType A[M][N])
{  int i,j; t.rows=M; t.cols=N; t.nums=0;
  for (i=0;i<M;i++)
  {  for (j=0;j<N;j++)
        if (A[i][j]!=0)
        {  t.data[t.nums].r=i;
           t.data[t.nums].c=j;
           t.data[t.nums].d=A[i][j];
           t.nums++;
        }
   }
}
```

按行序方式扫描所有元素

只存储非零元素

（2）三元组元素赋值：A[i][j]=x

分为两种情况：①将一个非0元素修改为另一个非0值，如A[5][6]=8。

修改元素

$A_{6\times7}=$

0	0	1	0	0	0	0
0	2	0	0	0	0	0
3	0	0	0	0	0	0
0	0	0	5	0	0	0
0	0	0	0	6	0	0
0	0	0	0	0	7	4

$A_{6\times7}=$

0	0	1	0	0	0	0
0	2	0	0	0	0	0
3	0	0	0	0	0	0
0	0	0	5	0	0	0
0	0	0	0	6	0	0
0	0	0	0	0	7	8

i	j	a_{ij}
0	2	1
1	1	2
2	0	3
3	3	5
4	4	6
5	5	7
5	6	4

i	j	a_{ij}
0	2	1
1	1	2
2	0	3
3	3	5
4	4	6
5	5	7
5	6	8

②将一个0元素修改为非0值。如A[3][5]=8

增加元素

$A_{6\times7}=$

$$\begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 & 0 & 0 & 0 \\ 3 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 5 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 6 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 7 & 4 \end{bmatrix} \Rightarrow A_{6\times7}= \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 & 0 & 0 & 0 \\ 3 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 5 & 0 & 8 & 0 \\ 0 & 0 & 0 & 0 & 6 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 7 & 4 \end{bmatrix}$$

i	j	a_{ij}
0	2	1
1	1	2
2	0	3
3	3	5
4	4	6
5	5	7
5	6	4

⇒

i	j	a_{ij}
0	2	1
1	1	2
2	0	3
3	3	5
3	**5**	**8**
4	4	6
5	5	7
5	6	8

算法如下：

```
bool Value(TSMatrix &t,ElemType x,int i,int j)
{  int k=0,k1;
   if (i>=t.rows || j>=t.cols)
      return false;                              //失败时返回false
  while (k<t.nums && i>t.data[k].r) k++;   //查找行
     while (k<t.nums && i==t.data[k].r
               && j>t.data[k].c)   k++;      //查找列
```

在t中按行、列号查找

```
  if (t.data[k].r==i && t.data[k].c==j)   //存在这样的元素
    t.data[k].d=x;
  else                  //不存在这样的元素时插入一个元素
  {  for (k1=t.nums-1;k1>=k;k1--)
     {  t.data[k1+1].r=t.data[k1].r;
        t.data[k1+1].c=t.data[k1].c;
        t.data[k1+1].d=t.data[k1].d;
     }
     t.data[k].r=i;t.data[k].c=j;t.data[k].d=x;
     t.nums++;
  }

  return true;        //成功时返回true
}
```

修改元素

增加元素

（3）将指定位置的元素值赋给变量　执行x=A[i][j]

先在三元组t中找到指定的位置，再将该处的元素值赋给x。

```
bool Assign(TSMatrix t,ElemType &x,int i,int j)
{  int k=0;
   if (i>=t.rows || j>=t.cols)
      return false;              //失败时返回false
  while (k<t.nums && i>t.data[k].r) k++;   //查找行
  while (k<t.nums && i==t.data[k].r
         && j>t.data[k].c)   k++;          //查找列
   if (t.data[k].r==i && t.data[k].c==j)
      x=t.data[k].d;
   else
      x=0;
   return true;                  //成功时返回true
}
```

在t中按行、列号查找

找到了非0的元素

没有找到：为0元素

（4）输出三元组

从头到尾扫描三元组t，依次输出元素值。

```
void DispMat(TSMatrix t)
{  int i;
   if (t.nums<=0) return;
   printf("\t%d\t%d\t%d\n",t.rows,t.cols,t.nums);
   printf("  ------------------\n");
   for (i=0;i<t.nums;i++)
      printf("\t%d\t%d\t%d\n",t.data[i].r,t.data[i].c,t.data[i].d);
}
```

（5）矩阵转置

对于一个$m\times n$的矩阵$A_{m\times n}$，其转置矩阵是一个$n\times m$的矩阵$B_{n\times m}$，满足$b_{i,j}=a_{j,i}$，其中$0\le i\le m-1$，$0\le j\le n-1$。

$$A_{6\times 7}=\begin{bmatrix}0&0&1&0&0&0&0\\0&2&0&0&0&0&0\\3&0&0&0&0&0&0\\0&0&0&5&0&0&0\\0&0&0&0&6&0&0\\0&0&0&0&0&7&4\end{bmatrix}$$

$$B_{7\times 6}=\begin{bmatrix}0&0&3&0&0&0\\0&2&0&0&0&0\\1&0&0&0&0&0\\0&0&0&5&0&0\\0&0&0&0&6&0\\0&0&0&0&0&7\\0&0&0&0&0&4\end{bmatrix}$$

i	j	a_{ij}
0	2	1
1	1	2
2	0	3
3	3	5
4	4	6
5	5	7
5	6	4

i	j	b_{ij}
0	2	3
1	1	2
2	0	1
3	3	5
4	4	6
5	5	7
6	5	4

一种非高效的算法：按第0、1、2、…、n-1列进行转换

i	j	a_{ij}
0	2	1
1	1	2
2	0	3
3	3	5
4	4	6
5	5	7
5	6	4

i	j	b_{ij}
0	2	3
1	1	2
2	0	1
3	3	5
4	4	6
5	5	7
6	5	4

矩阵转置

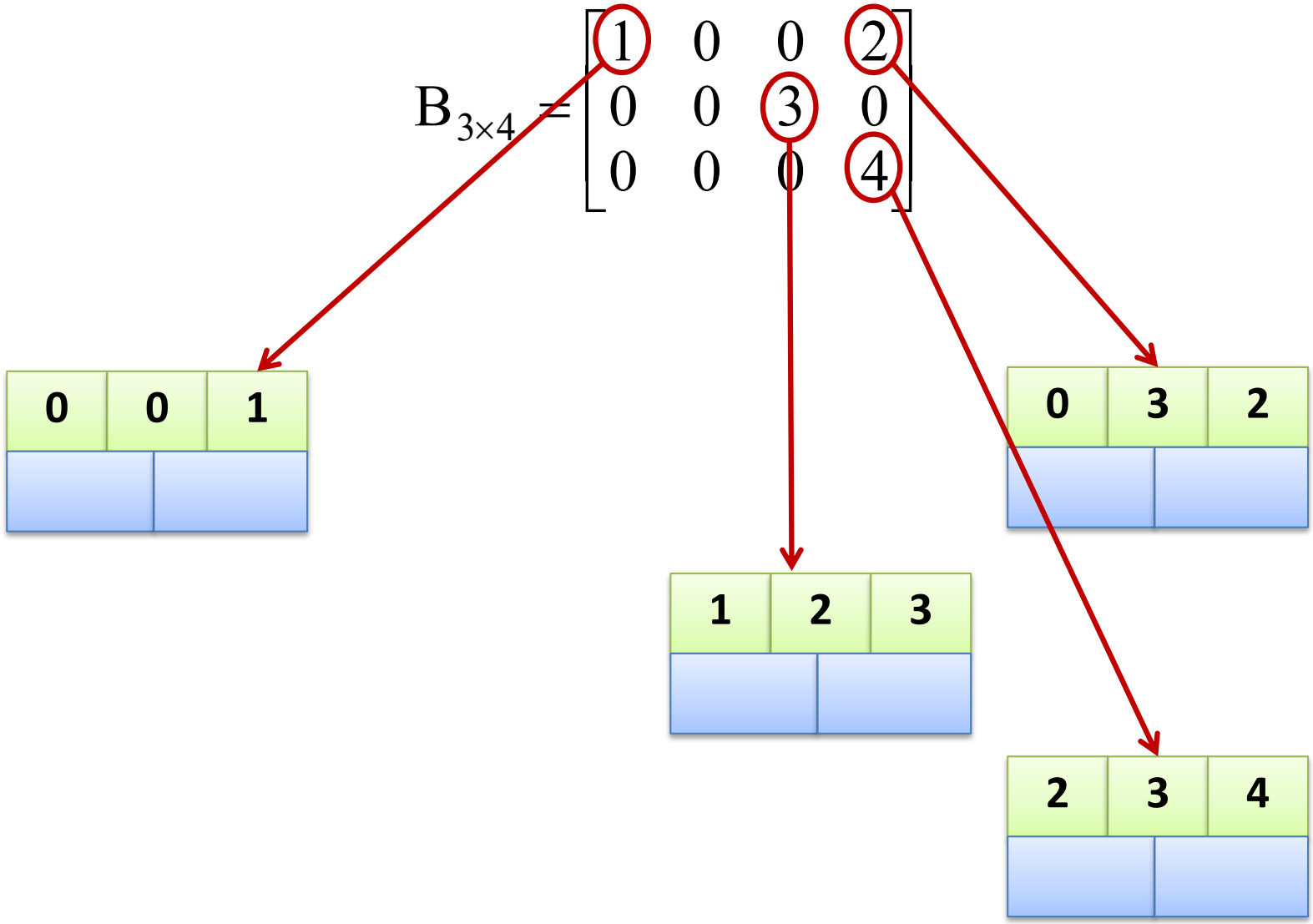
```
void TranTat(TSMatrix t,TSMatrix &tb)
{  int p,q=0,v;                          //q为tb.data的下标
   tb.rows=t.cols;  tb.cols=t.rows;  tb.nums=t.nums;
   if (t.nums!=0)                        //当存在非零元素时执行转置
   {
      for (v=0;v<t.cols;v++)             //tb.data[q]中记录以列序排列
         for (p=0;p<t.nums;p++)          //p为t.data的下标
            if (t.data[p].c==v)
            {  tb.data[q].r=t.data[p].c;
               tb.data[q].c=t.data[p].r;
               tb.data[q].d=t.data[p].d;
               q++;
            }
   }
}
```


按第0、1、2、…、*n*-1 列进行转换

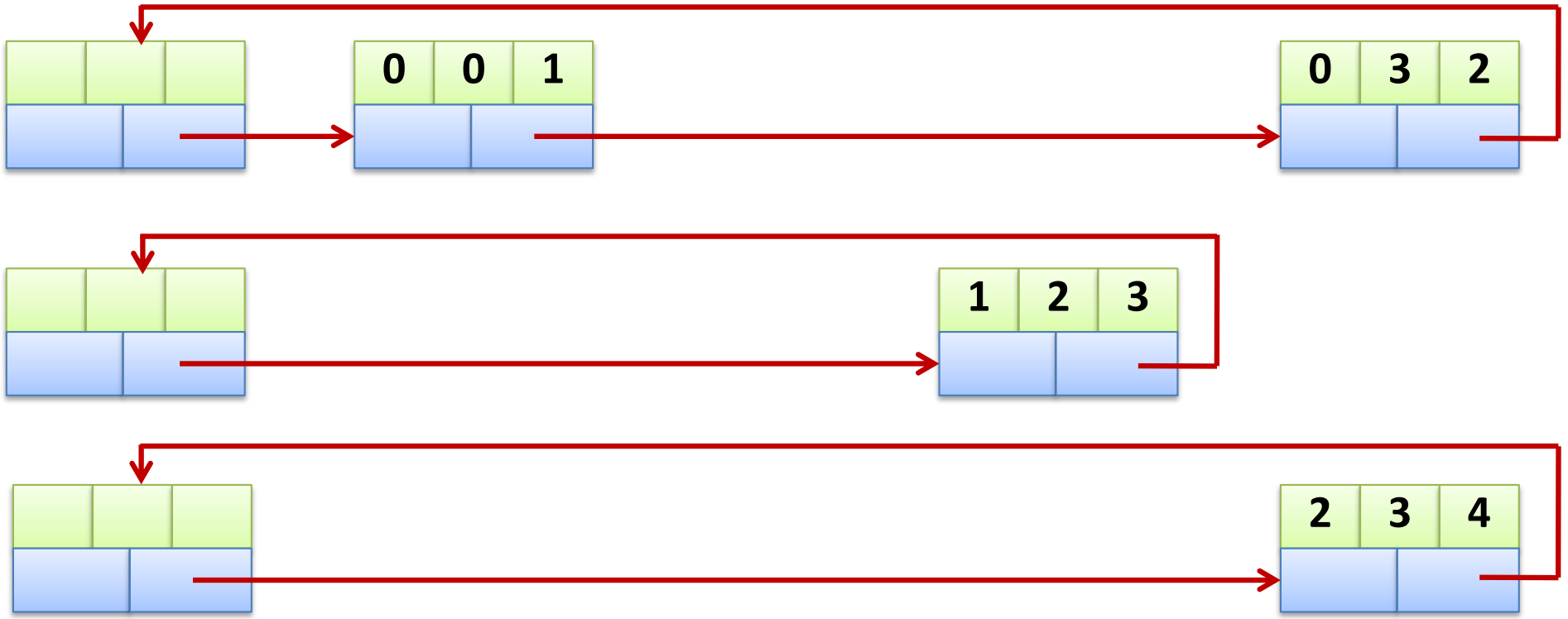
*m*行*n*列，*t*个非0元素，时间复杂度为O(*nt*)。

5.3.2 稀疏矩阵的十字链表表示

- 每个非零元素对应一个结点。

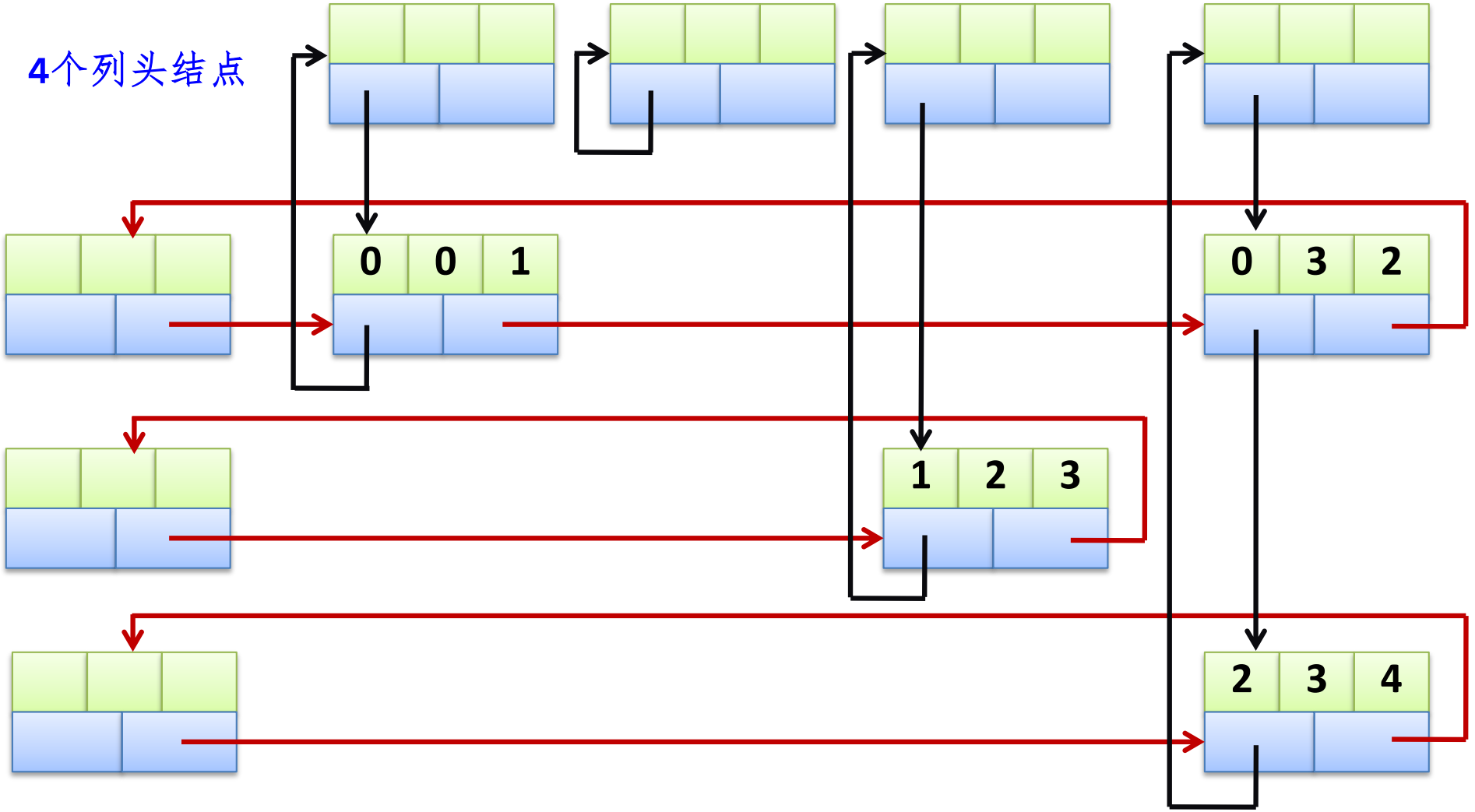


- 每行的所有结点链起来构成一个带行头结点的循环单链表。以 $h[i]$（$0 \leqslant i \leqslant m-1$）作为第$i$行的头结点。

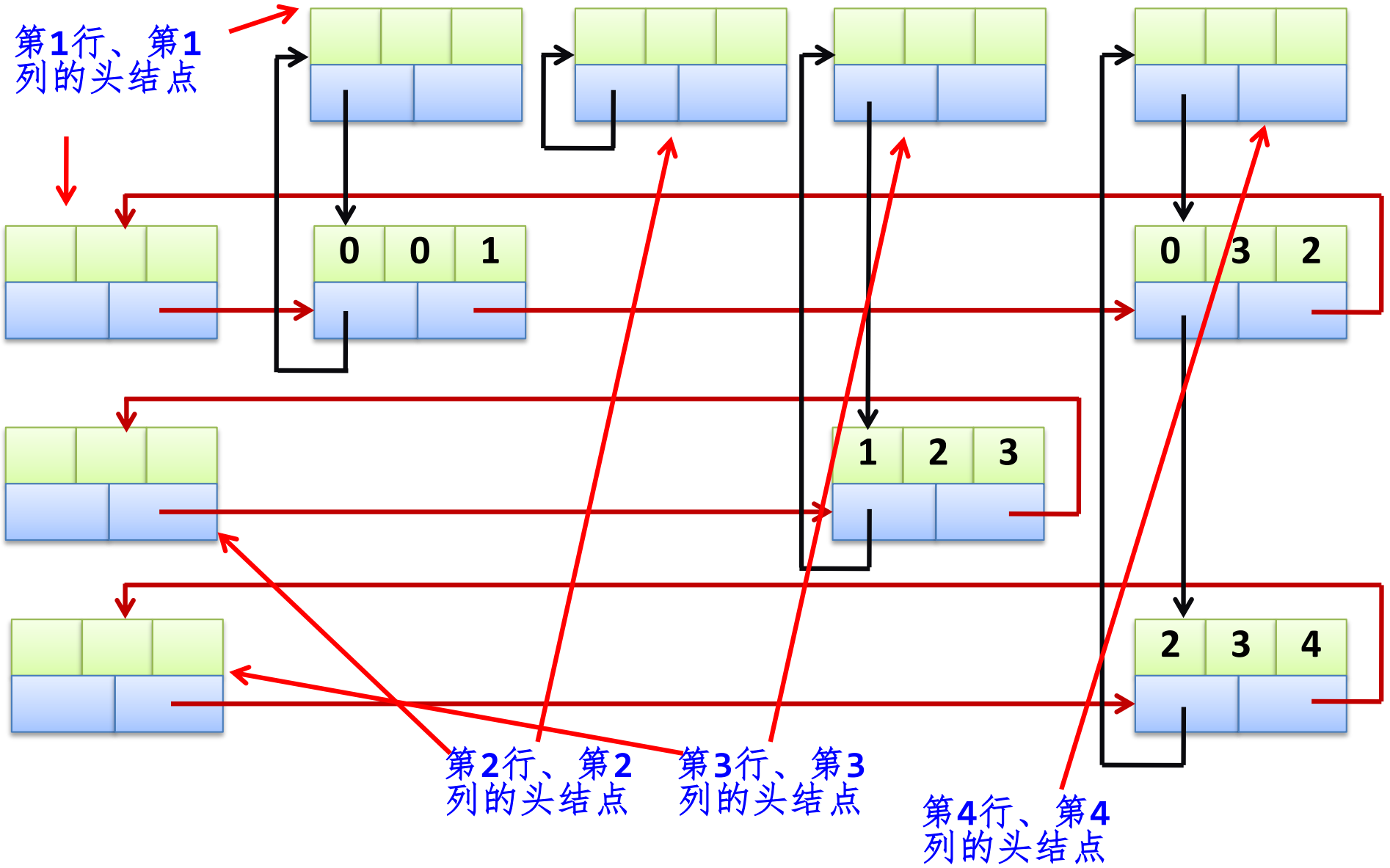


3个行头结点

- 每列的所有结点链起来构成一个带列头结点的循环单链表。
  以h[i]（0≤i≤m-1）作为第i列的头结点。

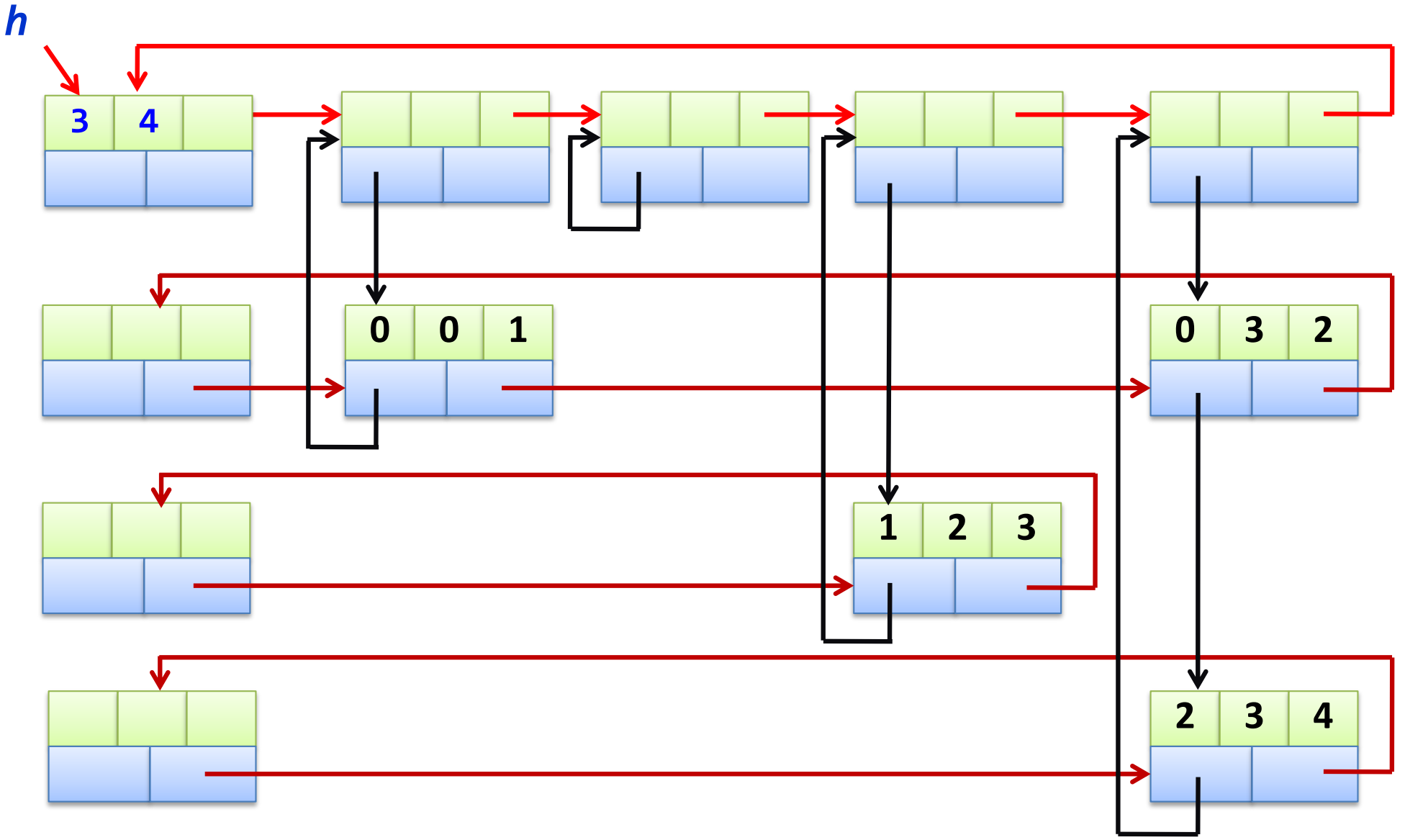


行、列头结点可以共享



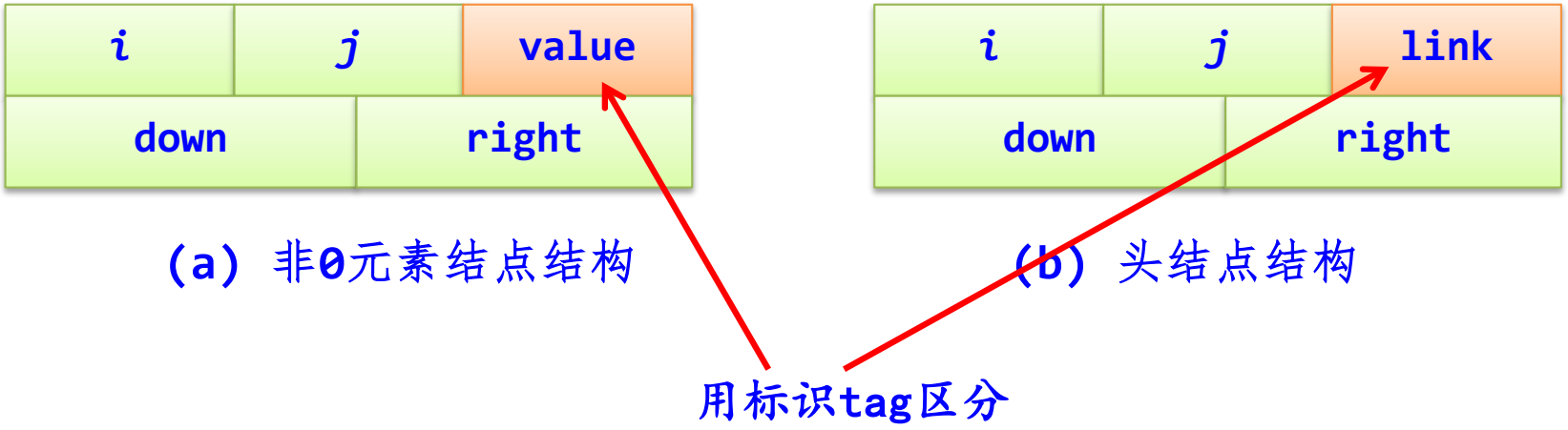
行、列头结点个数=MAX(*m*, *n*)

增加一个总头结点，并把所有行、列头结点链起来构成一个循环单链表

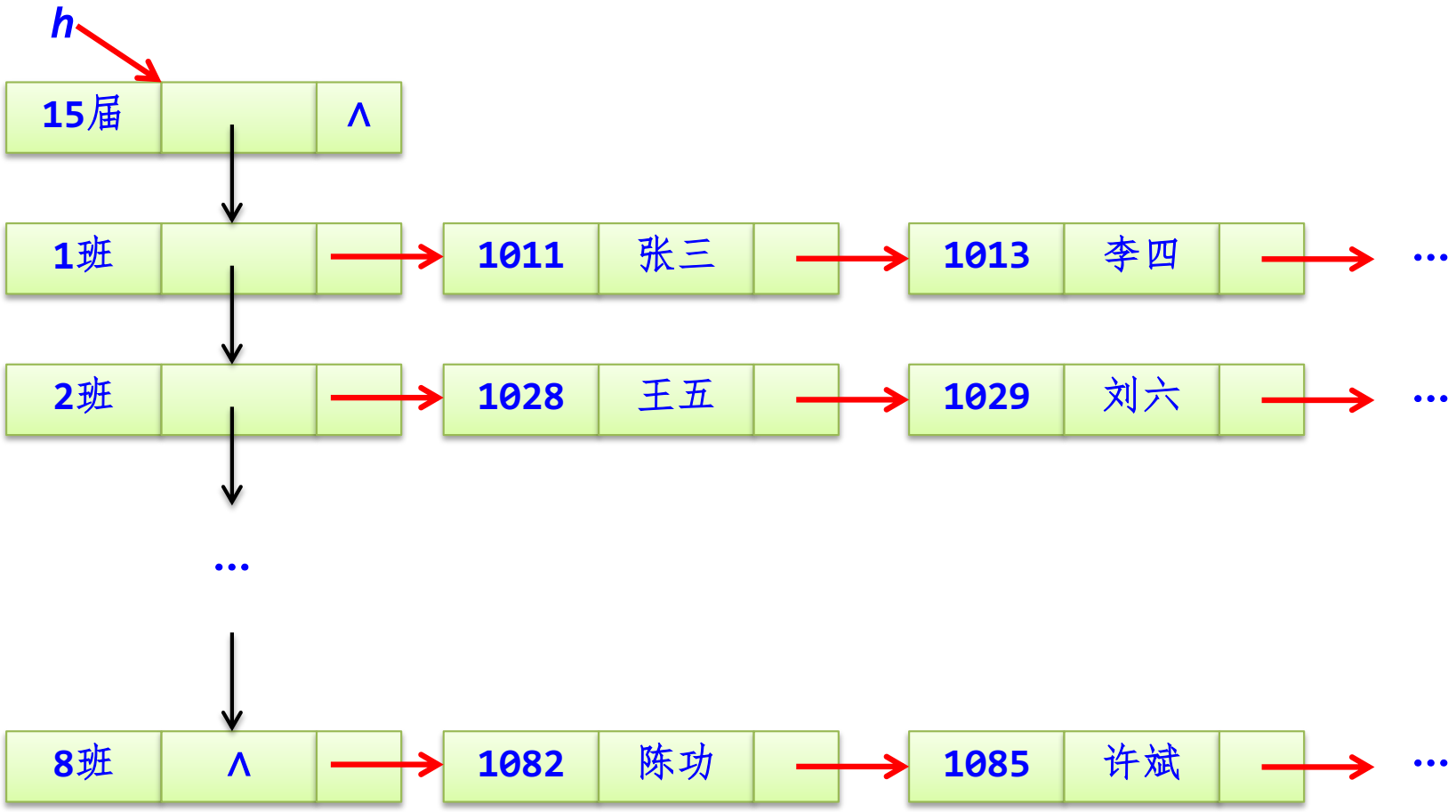


总的头结点个数=MAX(*m*,*n*)+1

为了统一，设计结点类型如下：



示例 十字链表的启示：设计存储某年级所有学生的存储结构。



通过*h*来唯一标识学生存储结构。

思考题

一个稀疏矩阵采用压缩后，和直接采用二维数组存储相比会失去（　）特性。

A.顺序存储　　B.随机存取

C.输入输出　　D.以上都不对

第6章 递 归

提纲

CONTENTS

6.1 什么是递归

6.2 递归算法的设计

6.3 递归算法转换为非递归算法

6.1 什么是递归

6.1.1 递归的定义

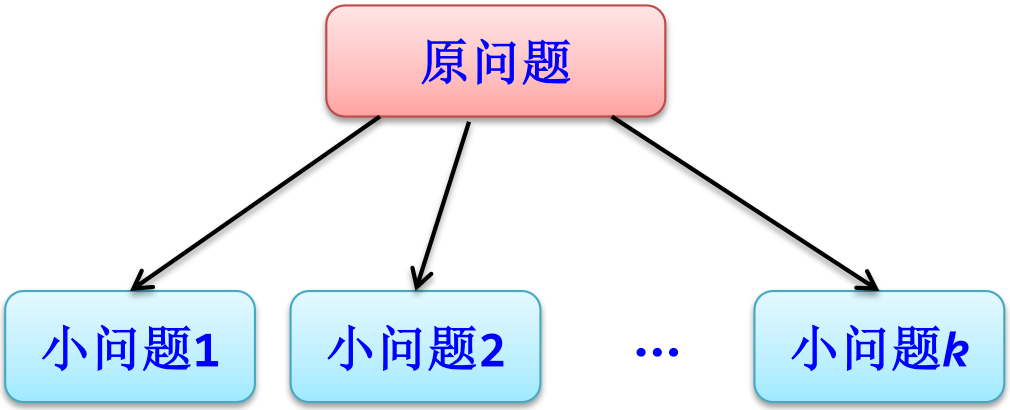
- 在定义一个算法时出现调用本算法的成分，称之为递归。
- 若调用自身，称之为直接递归。
- 若算法A调用算法B，而B又调用A，称之为间接递归。
- 在算法设计中，任何间接递归算法都可以转换为直接递归算法来实现，所以主要讨论直接递归。

【例6.1】以下是求*n*!（*n*为正整数）的递归算法。它属于什么类型的递归。

```
int fun(int n)
{  if (n==1)                    //语句1
      return 1;                 //语句2
   else                         //语句3
      return fun(n-1)*n;        //语句4
}
```

直接递归函数。

- 递归算法通常把一个大的复杂问题层层转化为一个或多个与原问题相似的规模较小的问题来求解。
- 递归策略只需少量的代码就可以描述出解题过程所需要的多次重复计算，大大减少了算法的代码量。



一般来说，能够用递归解决的问题应该满足以下3个条件：

- 需要解决的问题可以转化为一个或多个子问题来求解，而这些子问题的求解方法与原问题完全相同，只是在数量规模上不同。
- 递归调用的次数必须是有限的。
- 必须有结束递归的条件来终止递归。

6.1.2 何时使用递归

1．定义是递归的

有许多数学公式、数列等的定义是递归的。例如，求*n*!和Fibonacci（斐波那契）数列等。

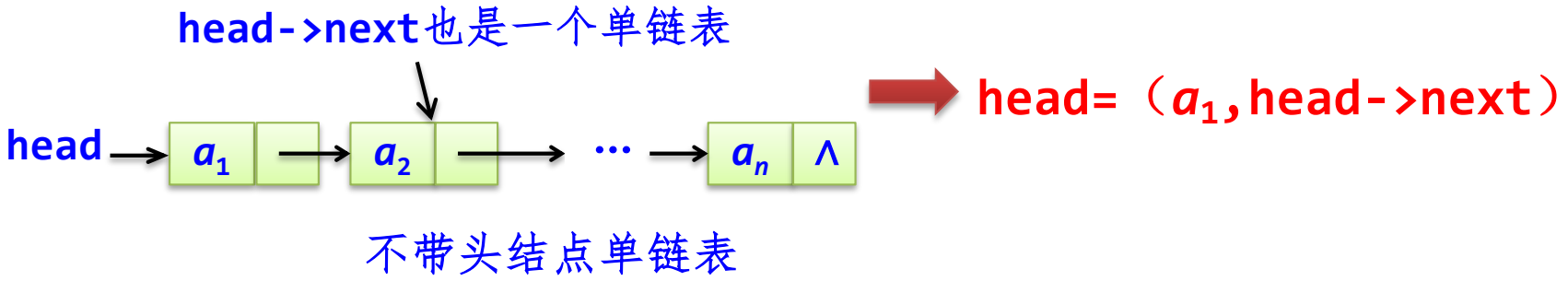
```
int Fib(int n)                    //求Fibonacci数列的第n项
{  if (n==1 || n==2)
       return 1;
   else
       return Fib(n-1)+Fib(n-2);
}
```

2．数据结构是递归的

有些数据结构是递归的。如单链表就是一种递归数据结构。

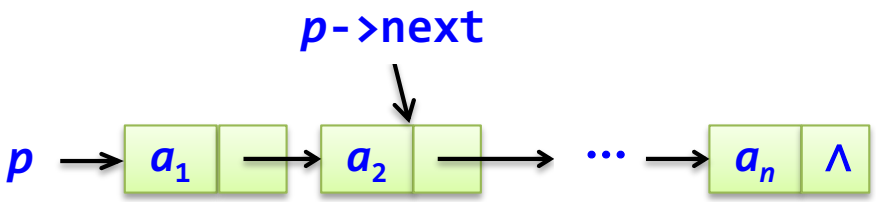
```
template <typename T>
class LinkNode                                  //单链表结点类
{
public:
   T data;                                      //存放数据元素
   LinkNode<T>* next;                           //指向下一个结点的指针域

   LinkNode():next(NULL) {}                     //构造函数
   LinkNode(T d):data(d),next(NULL) {}          //重载构造函数
};
```



不带头结点单链表

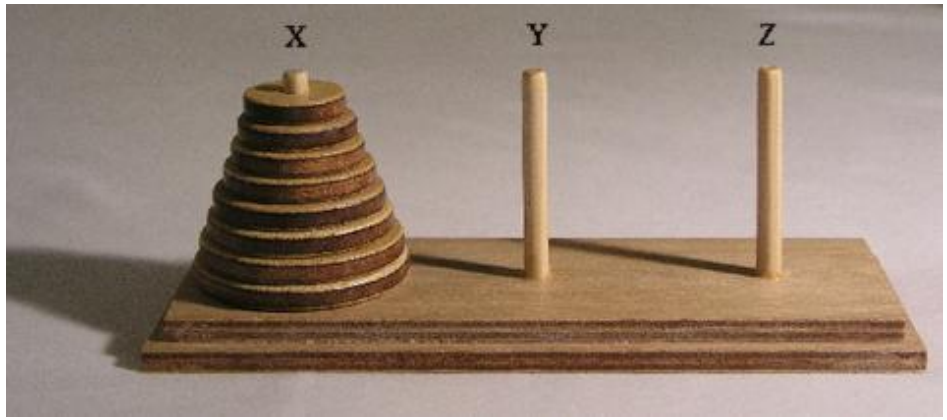
示例 求一个不带头结点单链表*p*中所有data成员（假设为int型）之和。




```
int Sum(LinkNode<int>* p)  //求不带头结点单链表p所有结点值之和
{  if (p==NULL)
      return 0;
   else
      return p->data+Sum(p->next);
}
```

3．问题的求解方法是递归的

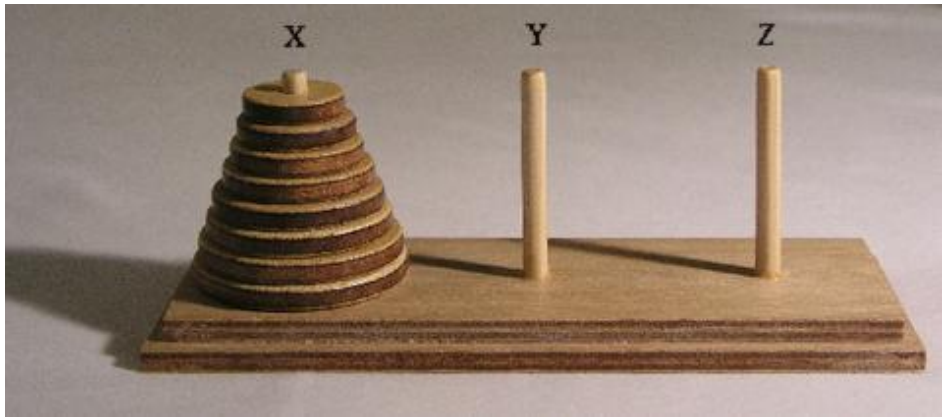
Hanoi问题



设Hanoi(*n*，*x*，*y*，*z*)表示将*n*个盘片从*x*塔座借助*y*塔座移动到*z*塔座上：

Hanoi(n，x，y，z)

```
Hanoi(n-1，x，z，y);
move(n，x，z)：将第n个圆盘从x移到z;
Hanoi(n-1，y，x，z)
```




```
void Hanoi(int n,char x,char y,char z)        //Hanoi递归算法
{  if (n==1)                                  //只有一个盘片的情况
      printf("将第%d个盘片从%c移动到%c\n",n,x,z);
   else                                       //有两个或多个盘片的情况
   {  Hanoi(n-1,x,z,y);
      printf("将第%d个盘片从%c移动到%c\n",n,x,z);
      Hanoi(n-1,y,x,z);
   }
}
```

$n=3$

1.一次只能移动一个磁盘
2.不能在较小的磁盘上放置较大的磁盘

6.1.3 递归模型

递归模型是递归算法的抽象，它反映一个递归问题的递归结构。

```
int fun(int n)
{  if (n==1)                    //语句1
      return 1;                 //语句2
   else                         //语句3
      return fun(n-1)*n;        //语句4
}
```

递归模型

$$f(n)=1 \qquad n=1$$
$$f(n)=n*f(n-1) \qquad n>1$$

$f(n)=1$ $n=1$ ← 递归出口

$f(n)=n*f(n-1)$ $n>1$ ← 递归体

- 一般地，一个递归模型是由递归出口和递归体两部分组成。
- 递归出口确定递归到何时结束，即指出明确的递归结束条件。
- 递归体确定递归求解时的递推关系。

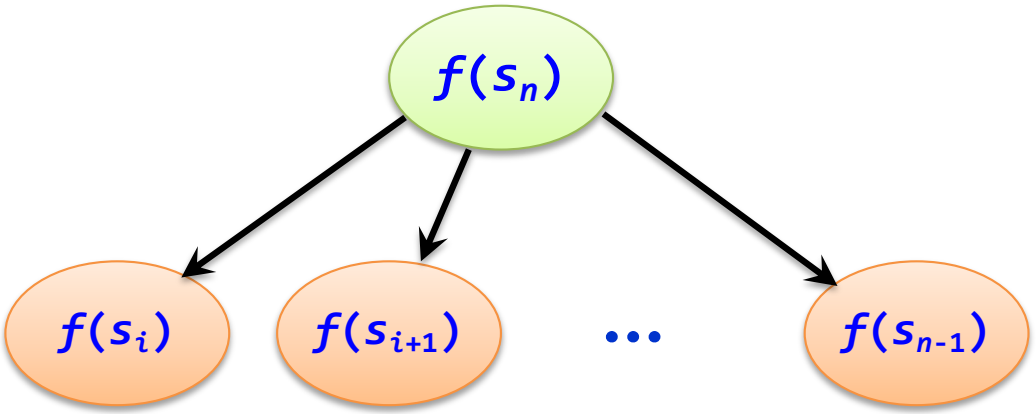
递归出口的一般格式如下：

$$f(s_1)=m_1$$

递归体的一般格式如下：

$$f(s_n)=g(f(s_i),\ f(s_{i+1}),\ \cdots,\ f(s_{n-1}),\ c_j,\ c_{j+1},\ \cdots,\ c_m)$$

非递归函数



大问题求解

转化

若干个相似子问题求解

6.1.4 递归与数学归纳法

采用数学归纳法证明$1+2+\cdots+n=n(n+1)/2$

- 当$n=1$时，左式=1，右式=$(1\times2)/2=1$，左右两式相等，等式成立。
- 假设当$n=k-1$时等式成立，有$1+2+\cdots+(k-1)=k(k-1)/2$。
- 当$n=k$时，左式=$1+2+\cdots+k=[1+2+\cdots+(k-1)+k=k(k-1)/2+k=k(k+1)/2$。

- 先考虑特殊情况。
- 然后假设$n=k-1$成立（第二数学归纳法是假设$n\leqslant k-1$均成立），再证明$n=k$时成立，即假设“小问题”成立，再推导出“大问题”成立。

- 先考虑特殊情况。 ← 递归出口
- 然后假设$n=k-1$成立（第二数学归纳法是假设$n \leqslant k-1$均成立），再证明$n=k$时成立，即假设“小问题”成立，再推导出“大问题”成立。 ← 递归体

- **递归出口相当于数学归纳法的特殊情况。**
- **递归体相当于数学归纳法的归纳步骤。**
- **区别：数学归纳法是一种论证方法，递归是算法和程序设计的一种实现技术。**
- **数学归纳法是递归求解问题的理论基础。**

6.1.5 递归的执行过程

简化的递归模型

$f(s_1)=m_1$

$f(s_n)=g(f(s_{n-1}),\ c_{n-1})$

求大问题$f(s_n)$：
分解（递推）和求值

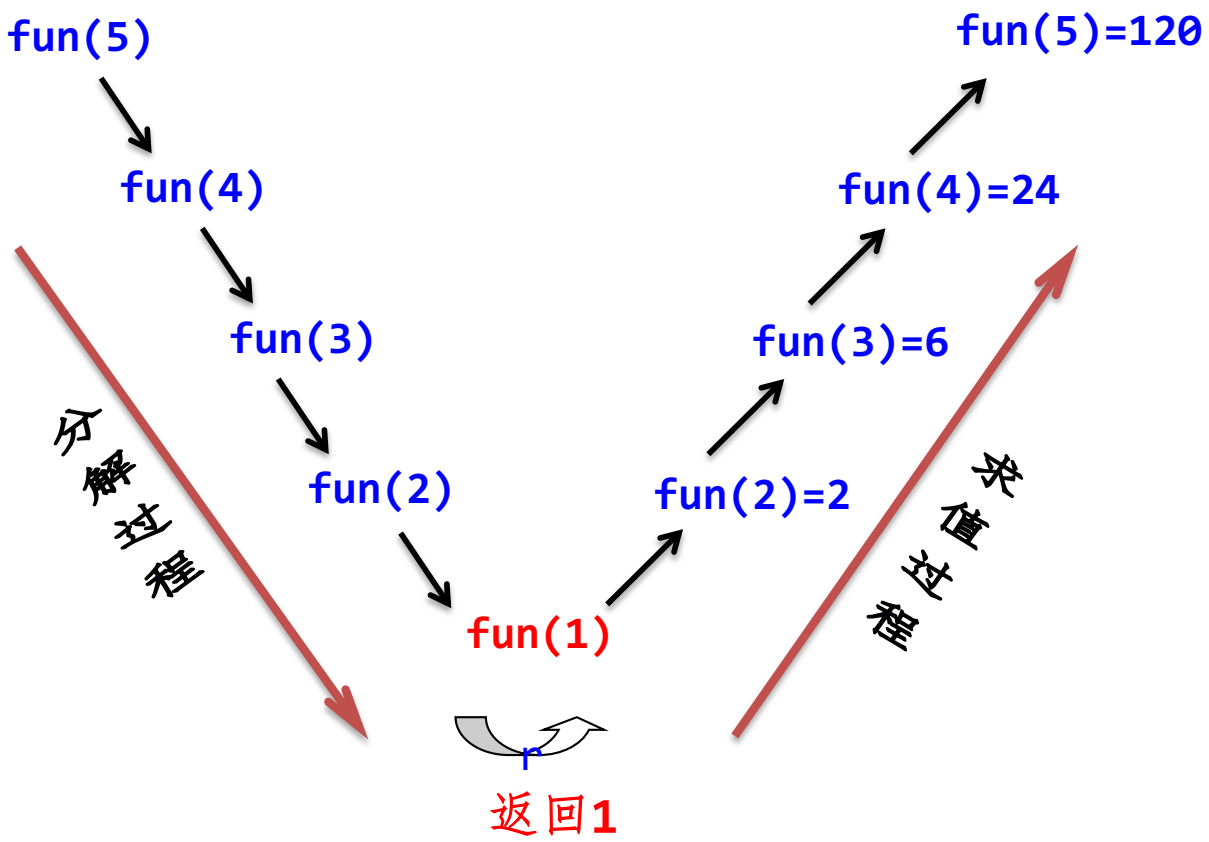
求$f(s_n)$的分解过程如下：

$f(s_n)$
↓
$f(s_{n-1})$
↓
…
↓
$f(s_2)$
↓
$f(s_1)$

- 遇到递归出口发生“质变”，原递归问题便转化成可以直接求解的问题。
- 求值过程：

$$f(s_1)=m_1$$
$$\downarrow$$
$$f(s_2)=g(f(s_1),c_1)$$
$$\downarrow$$
$$f(s_3)=g(f(s_2),c_2)$$
$$\downarrow$$
$$\cdots$$
$$\downarrow$$
$$f(s_n)=g(f(s_{n-1}),c_{n-1})$$

例如求5!。


fun(5)
fun(4)
fun(3)
fun(2)
fun(1)
fun(2)=2
fun(3)=6
fun(4)=24
fun(5)=120
分解过程
求值过程
n
返回1

系统内部如何执行递归算法

- 一个递归函数的调用过程类似于多个函数的嵌套的调用，只不过调用函数和被调用函数是同一个函数。
- 为了保证递归函数的正确执行，系统需设立一个工作栈。

（1）执行开始时，首先为递归调用建立一个工作栈，其结构包括值参、局部变量和返回地址。

（2）每次执行递归调用之前，把递归函数的值参和局部变量的当前值以及调用后的返回地址进栈。

（3）每次递归调用结束后，将栈顶元素出栈，使相应的值参和局部变量恢复为调用前的值，然后转向返回地址指定的位置继续执行。

示例

例如，有以下程序段：

```
int S(int n)
{  if (n<=0) return 0;
   else return S(n-1)+n;
}
int main()
{  print("%d\n",S(1));
   return 0;
}
```

程序执行时使用一个栈来保存调用过程的信息，这些信息用main()、S(0)和S(1)表示，那么自栈底到栈顶保存的信息的顺序是怎么样呢？

```
int S(int n)
{  if (n<=0) return 0;
   else return S(n-1)+n;
}
int main()
{  print("%d\n",S(1));
   return 0;
}
```

执行过程：

① 调用main()
② 调用S(1)
③ 调用S(0)
④ 从S(0)返回
⑤ 从S(1)返回
⑥ 从main()返回

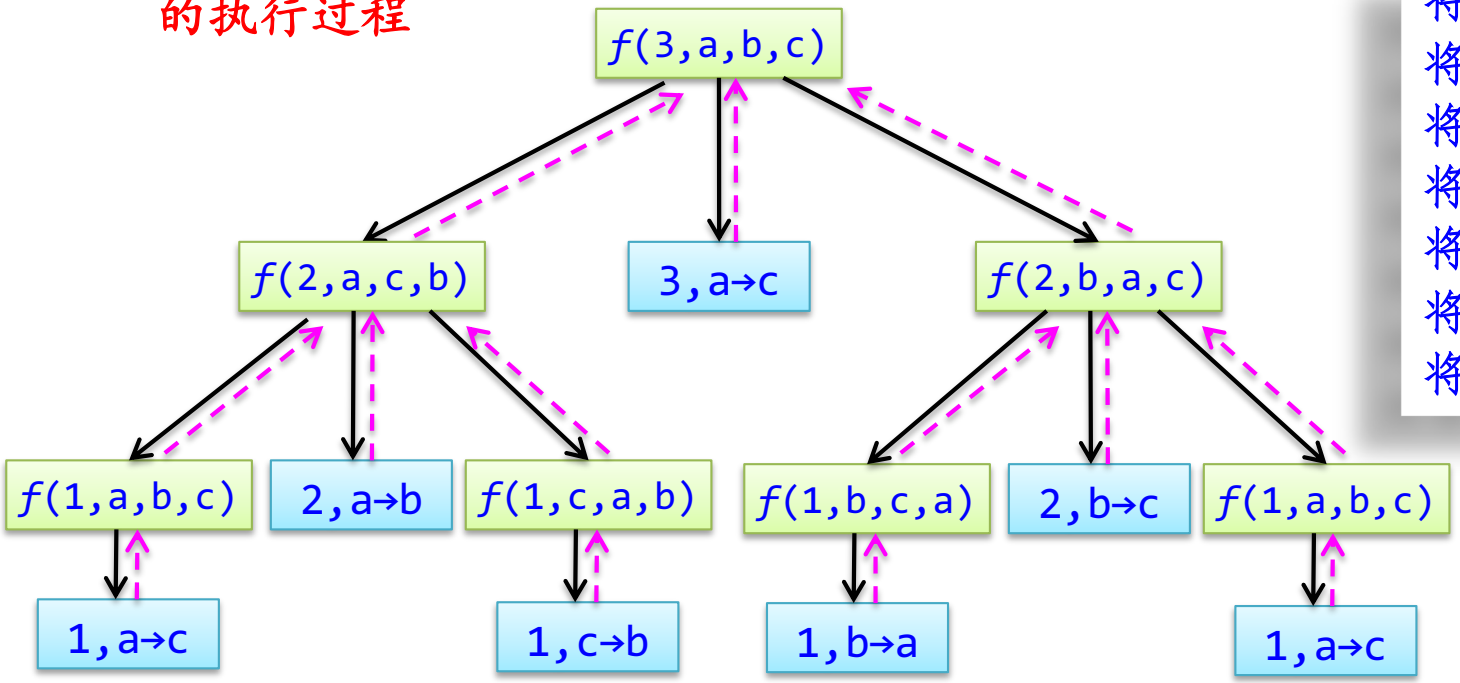
- 用递归算法的形参值表示**状态**，由于递归算法执行中系统栈保存了递归调用的值参、局部变量和返回地址。
- 所以在递归算法中一次递归调用后会自动恢复该次递归调用前的状态。

【例6.3】对例6.2的递归算法Hanoi()，给出调用Hanoi(3,a,b,c)时系统栈的变化过程。

```
void Hanoi(int n,char x,char y,char z)      //Hanoi递归算法
{  if (n==1)                                //只有一个盘片的情况
      printf("将第%d个盘片从%c移动到%c\n",n,x,z);
   else                                     //有两个或多个盘片的情况
   {  Hanoi(n-1,x,z,y);
      printf("将第%d个盘片从%c移动到%c\n",n,x,z);
      Hanoi(n-1,y,x,z);
   }
}
```

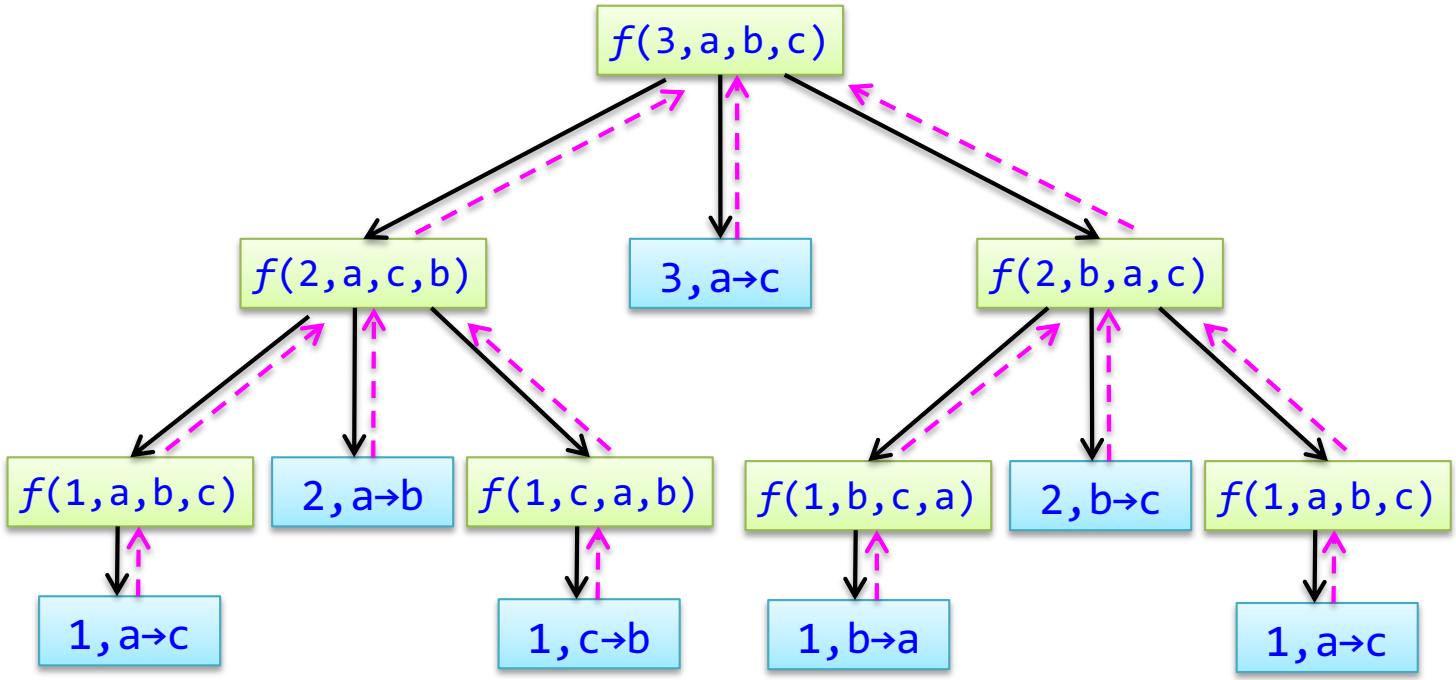
```
void Hanoi(int n,char x,char y,char z)      //Hanoi递归算法
{  if (n==1)                                //只有一个盘片的情况
      printf("将第%d个盘片从%c移动到%c\n",n,x,z);
   else                                     //有两个或多个盘片的情况
   {  Hanoi(n-1,x,z,y);
      printf("将第%d个盘片从%c移动到%c\n",n,x,z);
      Hanoi(n-1,y,x,z);
   }
}
```

Hanoi(3,a,b,c)
的执行过程

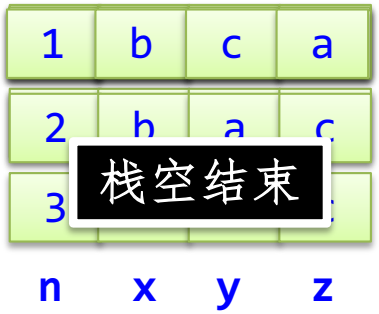


输出结果

将第1个盘片从a移动到c
将第2个盘片从a移动到b
将第1个盘片从c移动到b
将第3个盘片从a移动到c
将第1个盘片从b移动到a
将第2个盘片从b移动到c
将第1个盘片从a移动到c

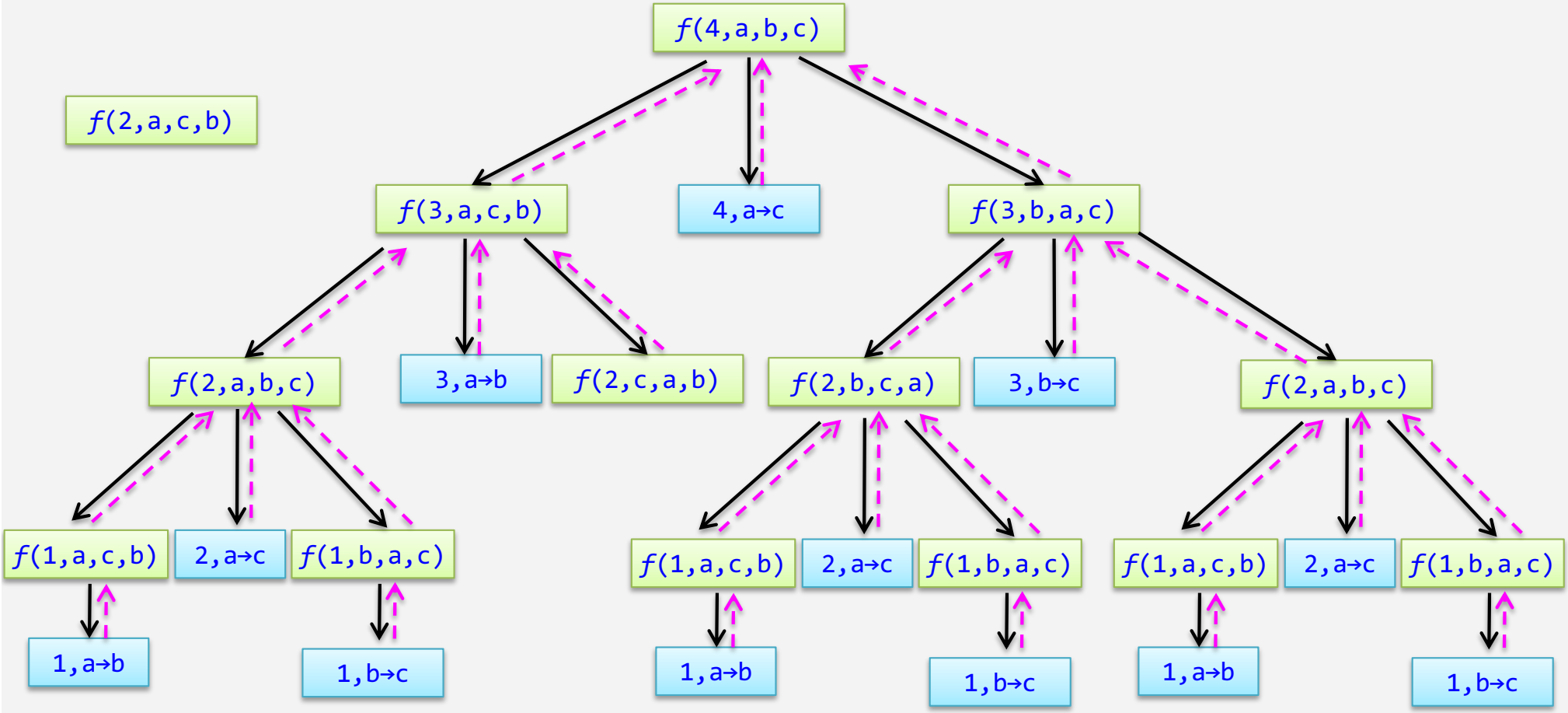


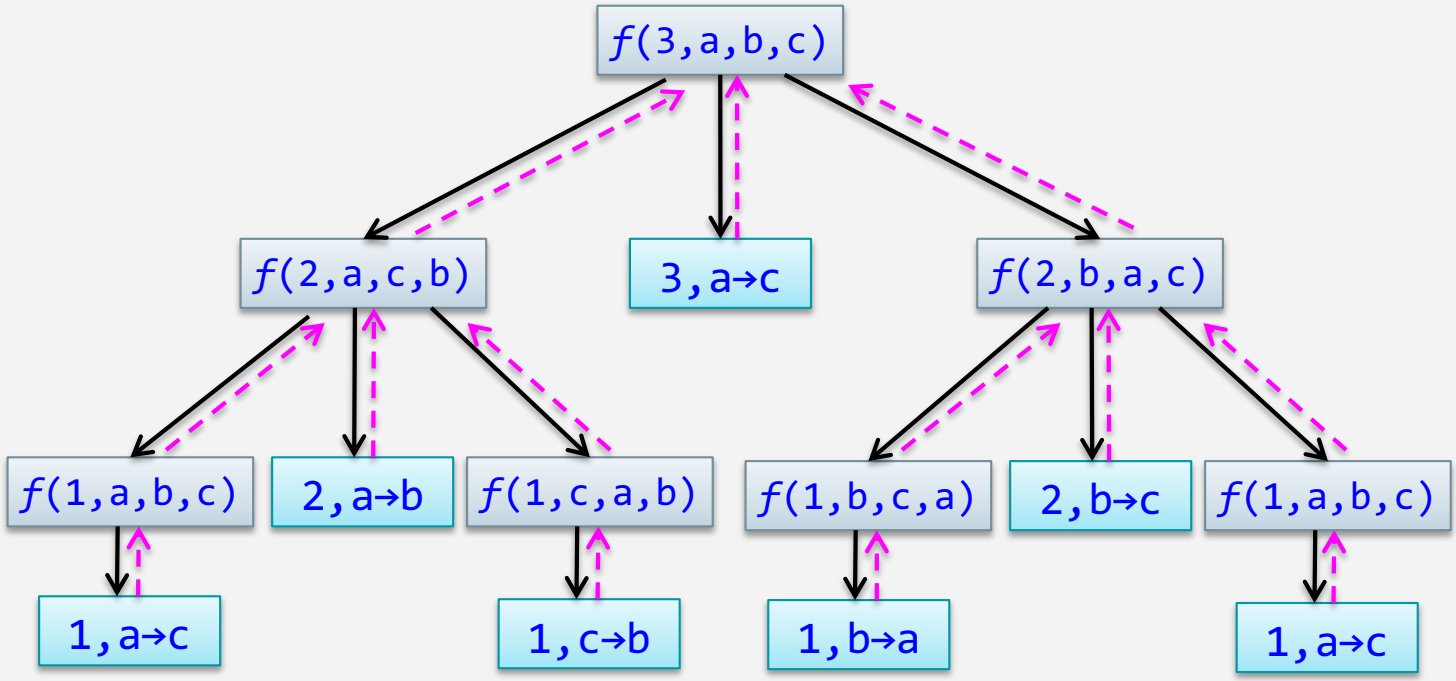
执行Hanoi(3,'a','b','c')时系统栈的变化过程



输出结果：

```
将第1个盘片从a移动到c
将第2个盘片从a移动到b
将第1个盘片从c移动到b
将第3个盘片从a移动到c
将第1个盘片从b移动到a
将第2个盘片从b移动到c
将第1个盘片从a移动到c
```


f(4,a,b,c)
f(2,a,c,b)
f(3,a,c,b)
4,a→c
f(3,b,a,c)
f(2,a,b,c)
3,a→b
f(2,c,a,b)
f(2,b,c,a)
3,b→c
f(2,a,b,c)
f(1,a,c,b)
2,a→c
f(1,b,a,c)
f(1,a,c,b)
2,a→c
f(1,b,a,c)
f(1,a,c,b)
2,a→c
f(1,b,a,c)
1,a→b
1,b→c
1,a→b
1,b→c
1,a→b
1,b→c



执行Hanoi(3,'a','b','c') 的过程

输出结果：

```
将第1个盘片从a移动到c
将第2个盘片从a移动到b
将第1个盘片从c移动到b
将第3个盘片从a移动到c
将第1个盘片从b移动到a
将第2个盘片从b移动到c
将第1个盘片从a移动到c
```

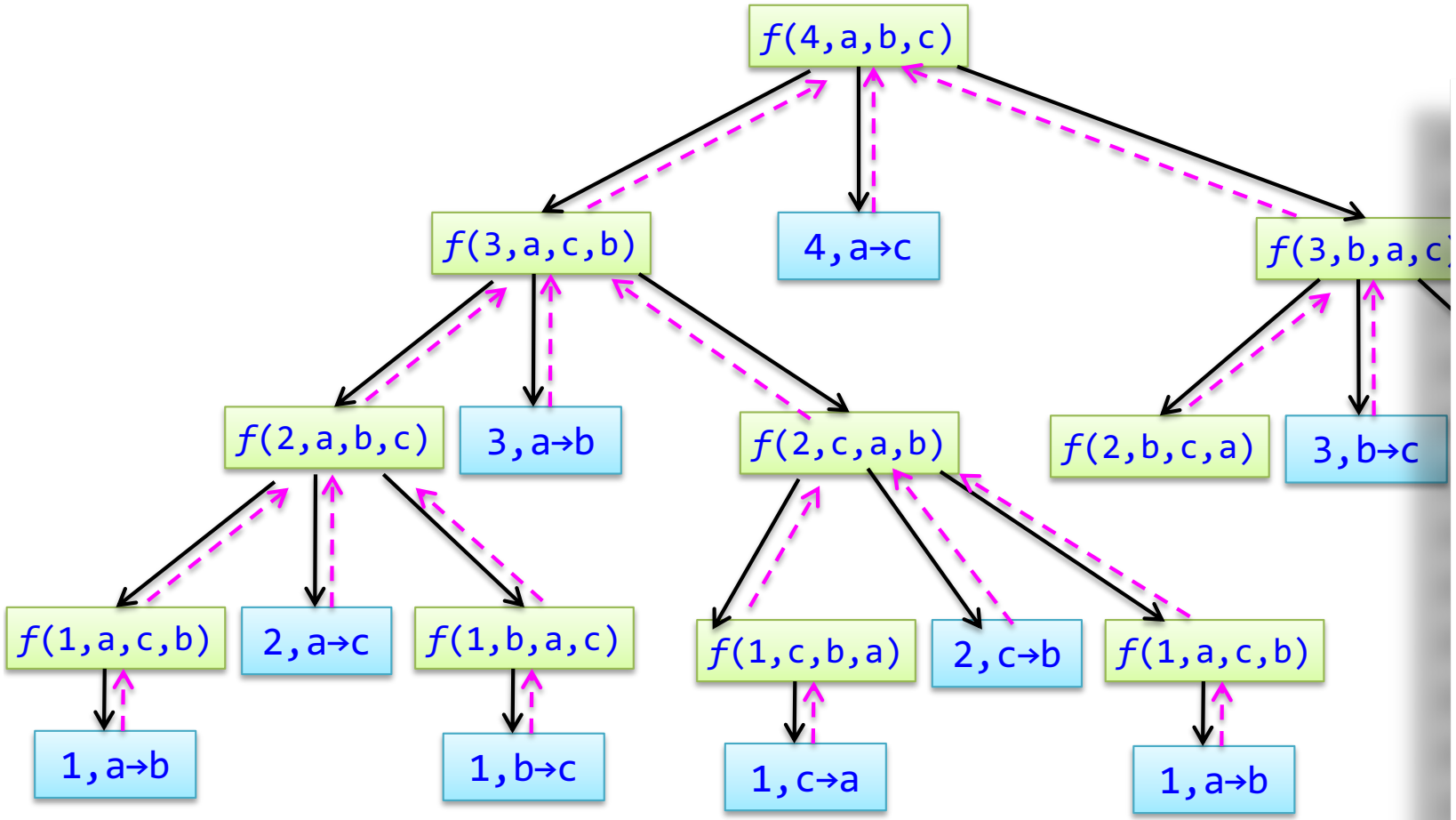
【练习】在Hanoi问题的递归算法中，当移动6个盘片时递归次数是多少？（移动多少次盘片？）

$m(n) = 1$ 当$n=1$

$m(n) = 2m(n-1)+1$ 当$n>1$

$$t(6) = 2t(5) + 1$$
$$= 2^2t(4) + 1 + 2$$
$$= 2^3t(3) + 1 + 2 + 2^2$$
$$= 2^4t(2) + 1 + 2 + 2^2 + 2^3$$
$$= 2^5t(1) + 1 + 2 + 2^2 + 2^3 + 2^4$$
$$= 1 + 2 + 2^2 + 2^3 + 2^4 + 2^5 = 2^6 - 1 = 63$$

【练习】画出Hanoi(4,x,y,z)的执行过程（调用树）并写出最终输出结果。




```
盘1: a --> b
盘2: a --> c
盘1: b --> c
盘3: a --> b
盘1: c --> a
盘2: c --> b
盘1: a --> b
盘4: a --> c
盘1: b --> c
盘2: b --> a
盘1: c --> a
盘3: b --> c
盘1: a --> b
盘2: a --> c
盘1: b --> c
```

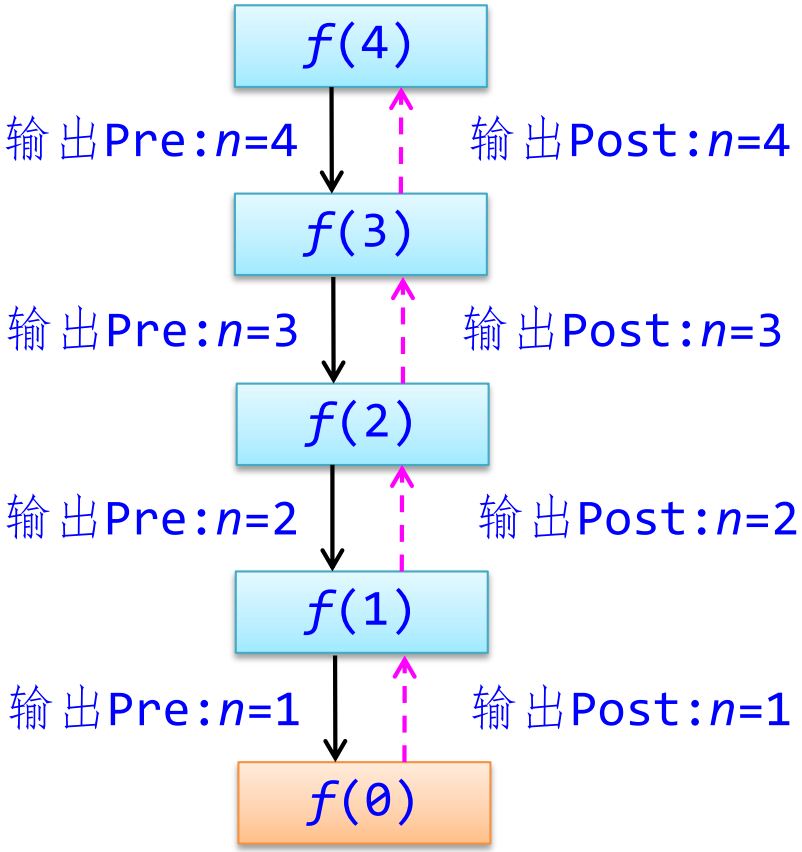
【例6.4】有以下递归算法，说明调用f(4)的过程。

```
void f(int n)                          //递归函数
{  if (n==0)                           //递归出口
      return;
   else                                //递归体
   {  printf("Pre:  n=%d\n",n);
      printf("执行f(%d)\n",n-1);
      f(n-1);
      printf("Post: n=%d\n",n);
   }
}
```

```
void f(int n)                          //递归函数
{  if (n==0)                           //递归出口
       return;
   else                                //递归体
   {  printf("Pre:  n=%d\n",n);
      printf("执行f(%d)\n",n-1);
      f(n-1);
      printf("Post: n=%d\n",n);
   }
}
```

调用*f*(4)的结果

```
Pre: n=4
执行f(3)                    //递归调用f(3)
Pre: n=3
执行f(2)                    //递归调用f(2)
Pre: n=2
执行f(1)                    //递归调用f(1)
Pre: n=1
执行f(0)                    //递归调用f(0)
Post:n=1                    //恢复f(0)调用前的n值
Post:n=2                    //恢复f(1)调用前的n值
Post:n=3                    //恢复f(2)调用前的n值
Post:n=4                    //恢复f(3)调用前的n值
```

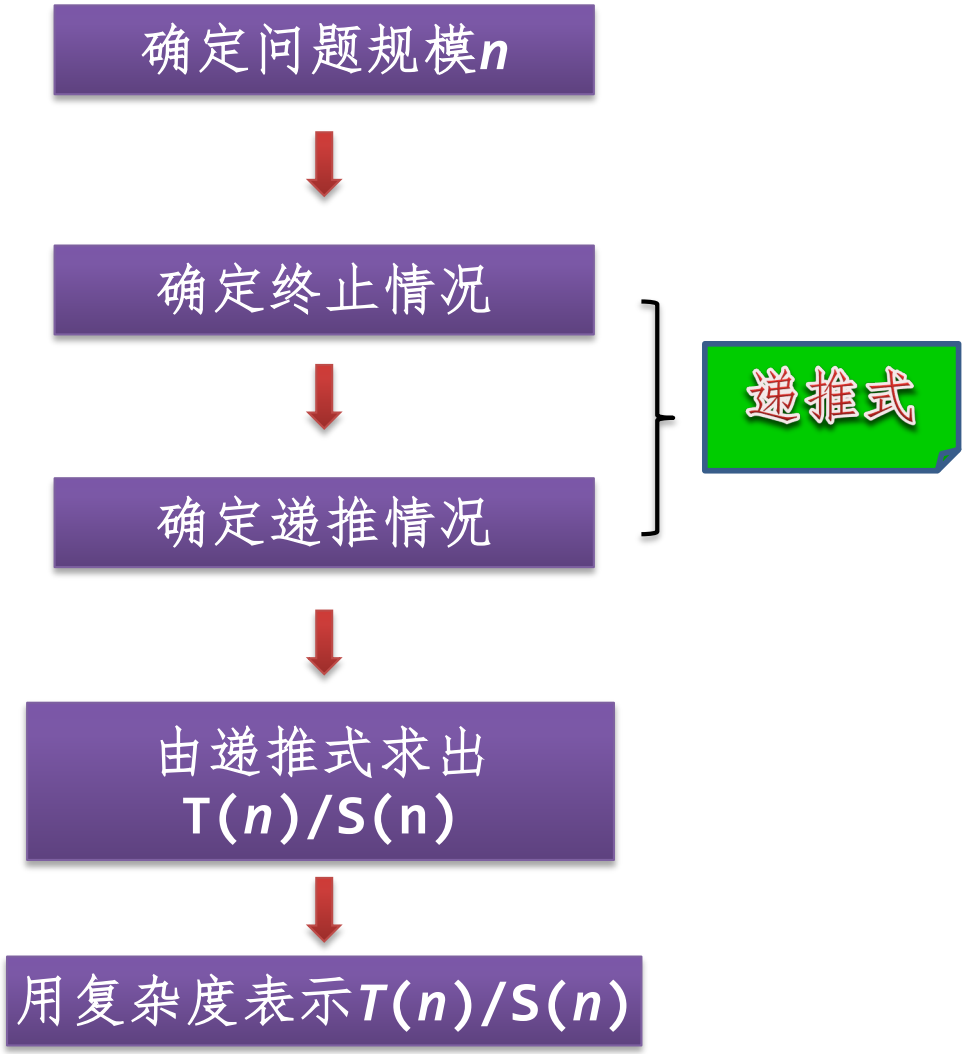

f(4)
输出Pre:n=4
输出Post:n=4
f(3)
输出Pre:n=3
输出Post:n=3
f(2)
输出Pre:n=2
输出Post:n=2
f(1)
输出Pre:n=1
输出Post:n=1
f(0)

调用f(4)的过程

5.1.6 递归算法的时空分析

- 递归算法执行过程不同于非递归算法，所以其时空分析也不同于非递归算法。
- 非递归算法分析是定长时空分析。
- 递归算法分析就是变长时空分析。

递归算法分析



1．递归算法的时间分析

```
void Hanoi(int n,char x,char y,char z)      //Hanoi递归算法
{  if (n==1)                                //只有一个盘片的情况
      printf("将第%d个盘片从%c移动到%c\n",n,x,z);
   else                                     //有两个或多个盘片的情况
   {  Hanoi(n-1,x,z,y);
      printf("将第%d个盘片从%c移动到%c\n",n,x,z);
      Hanoi(n-1,y,x,z);
   }
}
```

执行Hanoi(*n*,*x*,*y*,*z*)的时间复杂度为O(1)吗？

```
void Hanoi(int n,char x,char y,char z)        //Hanoi递归算法
{  if (n==1)                                  //只有一个盘片的情况
      printf("将第%d个盘片从%c移动到%c\n",n,x,z);
   else                                       //有两个或多个盘片的情况
   {  Hanoi(n-1,x,z,y);
      printf("将第%d个盘片从%c移动到%c\n",n,x,z);
      Hanoi(n-1,y,x,z);
   }
}
```

设大问题Hanoi(*n*，*x*，*y*，*z*)的执行时间为T(*n*)，则小问题Hanoi(*n*-1，*x*，*y*，*z*)的执行时间为T(*n*-1)。递推式：

T(*n*)=1	当*n*=1时
T(*n*)=2T(*n*-1)+1	当*n*>1时

$T(n)=1$ 当$n=1$时

$T(n)=2T(n-1)+1$ 当$n>1$时

$$T(n)=2T(n-1)+1=2(2T(n-2)+1)+1$$

$$=2^2T(n-2)+2+1=2^2(2T(n-3)+1)+2+1$$

$$=2^3T(n-3)+2^2+2+1$$

$$=\cdots$$

$$=2^{n-1}T(1)+2^{n-2}+\cdots+2^2+2+1$$

$$=2^n-1$$

$$=O(2^n)$$

1．递归算法的时间分析

【例1-9：p22】　有如下递归算法：

```
void fun(int a[], int n, int k) //数组a共有n个元素
{  int i;
   if (k==n-1)
     for (i=0;i<n;i++)
        printf("%d\n", a[i]); //执行n次
   else
   {  for (i=k;i<n;i++)
         a[i]=a[i]+i*i;       //执行n-k次
      fun(a, n, k+1);
   }
 }
```

调用上述算法的语句为fun(*a*, *n*, 0)，求其时间复杂度。

递归算法：

```
void fun(int a[], int n, int k) //数组a共有n个元素
{  int i;
   if (k==n-1)
     for (i=0;i<n;i++)
        printf("%d\n", a[i]); //执行n次
   else
   {  for (i=k;i<n;i++)
         a[i]=a[i]+i*i;       //执行n-k次
      fun(a, n, k+1);
   }
 }
```

含一重循环

fun(*a*, *n*, 0)的时间复杂度为O(*n*)。 ⟸ 错误

解：设fun(*a*, *n*, 0)的执行时间为T(*n*)，fun(*a*, *n*, *k*)的执行时间为$T_1(n, k)$ ⇨ $T(n)=T_1(n, 0)$。

```
void fun(int a[], int n, int k) //数组a共有n个元素
                                //执行时间为T1(n, k)
{  int i;
   if (k==n-1)
     for (i=0;i<n;i++)
        printf("%d\n", a[i]); //执行n次
   else
   {  for (i=k;i<n;i++)
         a[i]=a[i]+i*i;       //执行n-k次
      fun(a, n, k+1);         //执行时间为T1(n, k+1)
   }
 }
```

解：设fun(*a*, *n*, 0)的执行时间为T(*n*)，fun(*a*, *n*, *k*)的执行时间为$T_1(n, k)$ ⇨ $T(n)=T_1(n, 0)$。

由fun()递归算法可知：

$$T_1(n, k) = n \quad \text{当} k=n-1\text{时}$$
$$T_1(n, k) = (n-k)+T_1(n, k+1) \quad \text{其他情况}$$

则

$$T(n) = T_1(n, 0) = n+T_1(n, 1) = n+(n-1)+T_1(n, 2)$$
$$= \cdots = n+(n-1)+\cdots+2+T_1(n, n-1)$$
$$= n+(n-1)+\cdots+2+n$$
$$= O(n^2)$$

所以调用fun(*a*, *n*, 0)的时间复杂度为$O(n^2)$。

1．代入法

代入法：先“猜”一个结论，然后用数学归纳法证明

【例1】给定递推式 $T(n)=2T(n/2)+n$，求解$T(n)$。

我们首先猜测结论是$T(n)=O(n\log n)$。

证明：采用数学归纳法，假设结论对所有 $m<n$ 成立，即存在常数c，使得$T(m)\leq c\cdot m\log m$。下面要证明该不等式也对n成立。

因为 $m=n/2<n$，所以有 $T(n/2)\leq c\cdot(n/2)\log(n/2)$。将此不等式代入给定的递推式，得到

$$T(n)=2T(n/2)+n\leq 2[c\cdot(n/2)\log(n/2)]+n$$

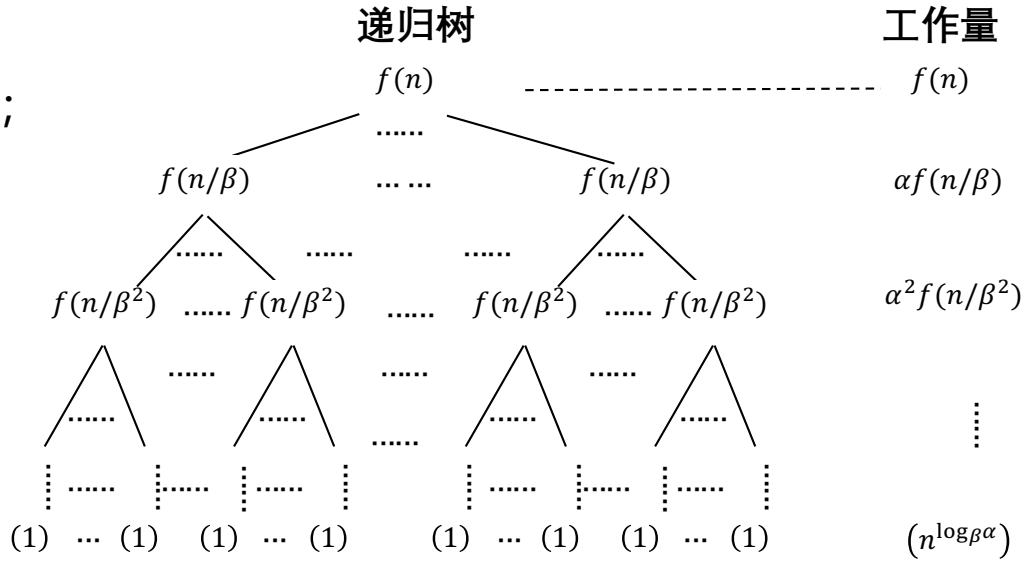
$$=c\cdot n\log n+n(1-c)\leq c\cdot n\log n$$

对任意常数$c\geq 1$成立。这就完成了数学归纳法的证明。

2．归纳树法

递归树法：用递归树将递归过程展开，看到递归每一层的工作量，然后对整体工作量进行评估

根结点对应$T(n)$中“合”的工作量$f(n)$；根结点有α个子树，每棵子树对应一个递归调用$T(n/\beta)$。树右侧展示了每一层“合”的工作量。递归的层数是$\log_\beta n$，最后一层有$\alpha^{\log_\beta n} = n^{\log_\beta \alpha}$个$(1)$，所以总工作量就是所有递归层工作量的总和：



$$T(n) = \sum_{j=0}^{\log_\beta n - 1} \alpha^j f(n/\beta^j) + \left(n^{\log_\beta \alpha}\right)$$

用递归树法大致估算工作量，用代入法严格证明结论。

3.主定理法

【主定理】设$T(n)$ 为定义在非负整数 n 上的函数，满足递推式 $T(n)=\alpha T(n/\beta)+f(n)$，其中常数$\alpha \geq 1$，$\beta > 1$，$f(n)$为给定函数。则有下列结论：

1. 若存在常数$\varepsilon > 0$ 使得$f(n)=O(n^{\log_\beta \alpha-\varepsilon})$，则 $T(n)=\left(n^{\log_\beta \alpha}\right)$。
2. 若 $f(n)=\left(n^{\log_\beta \alpha}\right)$，则 $T(n)=\left(n^{\log_\beta \alpha}\log n\right)$。
3. 若存在常数$\varepsilon > 0$ 使得$f(n)=\Omega(n^{\log_\beta \alpha+\varepsilon})$；并且存在常数 $c<1$，使得对充分大的n 有不等式 $\alpha f(n/\beta)<cf(n)$，则 $T(n)=\Theta(f(n))$。

可以看到关于$T(n)$的结论取决于$f(n)$和函数$n^{\log_\beta \alpha}$谁更占优，占优势的一方就决定了$T(n)$；当两者势均力敌时，结论中会多出一项$\log n$。

3.主定理法

$f(n)$和函数$n^{\log_\beta \alpha}$谁更占优，就决定了$T(n)$；

当两者势均力敌时，结论中会多出一项$\log n$。

【例1】给定递推式 $T(n) = 4\,T\left(\frac{n}{2}\right) + n$ ，求解$T(n)$。

【例2】给定递推式 $T(n) = 3\,T\left(\frac{n}{2}\right) + n$ ，求解$T(n)$。

【例3】给定递推式 $T(n) = 2\,T\left(\frac{n}{2}\right) + n$ ，求解$T(n)$。

3.主定理法

【主定理2】给定常数$\alpha \geq 1$，$\beta > 1$，$p \geq 0$，递推式$T(n) = \alpha T(n/\beta) + \Theta(n^k \log^p n)$的解有如下结论：

1. 若$\alpha > \beta^k$，则$T(n) = O\left(n^{\log_\beta \alpha}\right)$。
2. 若$\alpha = \beta^k$，则$T(n) = O(n^k \log^{p+1} n)$。
3. 若$\alpha < \beta^k$，则$T(n) = O(n^k \log^p n)$。

2．递归算法的空间分析

```
void Hanoi(int n,char x,char y,char z)      //Hanoi递归算法
{  if (n==1)                                //只有一个盘片的情况
      printf("将第%d个盘片从%c移动到%c\n",n,x,z);
   else                                     //有两个或多个盘片的情况
   {  Hanoi(n-1,x,z,y);
      printf("将第%d个盘片从%c移动到%c\n",n,x,z);
      Hanoi(n-1,y,x,z);
   }
}
```

执行Hanoi(*n*,*x*,*y*,*z*)的空间复杂度为O(1)吗？

```
void Hanoi(int n,char x,char y,char z)      //Hanoi递归算法
{  if (n==1)                                //只有一个盘片的情况
      printf("将第%d个盘片从%c移动到%c\n",n,x,z);
   else                                     //有两个或多个盘片的情况
   {  Hanoi(n-1,x,z,y);
      printf("将第%d个盘片从%c移动到%c\n",n,x,z);
      Hanoi(n-1,y,x,z);
   }
}
```

设大问题Hanoi(*n*，*x*，*y*，*z*)的占用空间为S(*n*)，则小问题Hanoi(*n*-1，*x*，*y*，*z*)的占用空间为S(*n*-1)。递推式：

S(*n*)=1	当*n*=1时
S(*n*)=S(*n*-1)+1	当*n*>1时

$S(n)=1$ 当$n=1$时

$S(n)=S(n-1)+1$ 当$n>1$时

$$
\begin{aligned}
S(n) &= S(n-1)+1 \\
&= S(n-2)+1+1=S(n-2)+2 \\
&= \cdots \\
&= S(1)+(n-1)=1+(n-1) \\
&= n \\
&= O(n)
\end{aligned}
$$

2．递归算法的空间分析

【例1-11：p23】有如下递归算法，分析调用fun(*a*，*n*，0)的空间复杂度。

```
void fun(int a[], int n, int k) //数组a共有n个元素
{  int i;
   if (k==n-1)
     for (i=0;i<n;i++)
        printf("%d\n", a[i]); //执行n次
   else
   {  for (i=k;i<n;i++)
         a[i]=a[i]+i*i;       //执行n-k次
      fun(a, n, k+1);
   }
 }
```

递归算法：

```
void fun(int a[], int n, int k) //数组a共有n个元素
{  int i;
   if (k==n-1)
     for (i=0;i<n;i++)
        printf("%d\n", a[i]); //执行n次
   else
   {  for (i=k;i<n;i++)
         a[i]=a[i]+i*i;       //执行n-k次
      fun(a, n, k+1);
   }
 }
```

仅仅定义了一个临时变量i

fun(*a*, *n*, 0)的空间复杂度为O(1)。 ⟸ 错误

解：设fun(*a*, *n*, 0)的空间为S(*n*)，fun(*a*, *n*, *k*)的空间为$S_1(n, k)$ ⇨ $S(n) = S_1(n, 0)$。

由fun()递归算法可知：

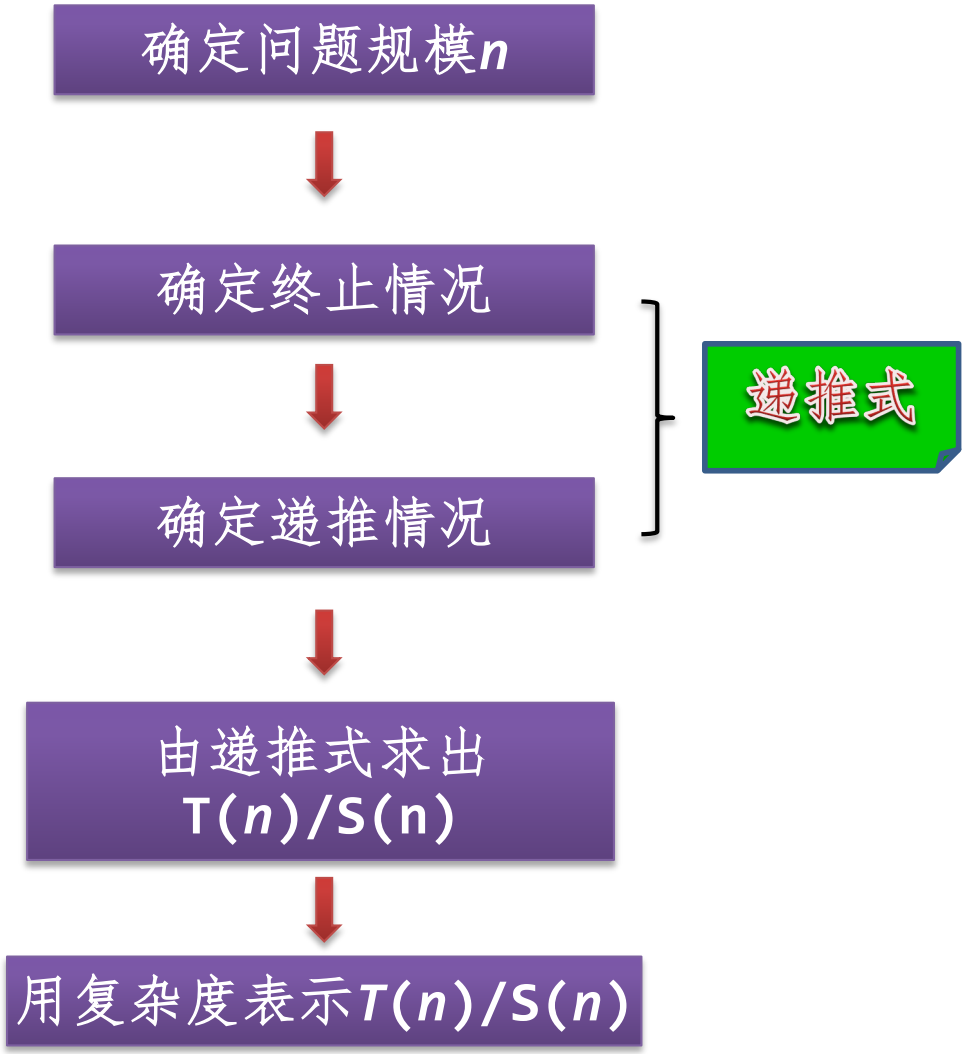
$S_1(n, k) = 1$	当$k=n-1$时
$S_1(k) = 1+S_1(n, k+1)$	其他情况

则：

$$S(n) = S_1(n, 0) = 1+S_1(n, 1) = 1+1+S_1(n, 2)$$
$$= \cdots = \underbrace{1 + 1 + \cdots + 1}_{n\text{个}1} = O(n)$$

所以调用fun(*a*, *n*, 0)的空间复杂度为$O(n)$。

递归算法分析



6.2 递归算法的设计

6.2.1 递归算法设计的步骤

- 设计求解问题的递归模型。
- 转换成对应的递归算法。

求递归模型的步骤如下：

（1）对原问题$f(s)$进行分析，称为“大问题”，假设出合理的“小问题”$f(s')$；

数学归纳法

（2）假设$f(s')$是可解的，在此基础上确定$f(s)$的解，即给出$f(s)$与$f(s')$之间的关系 ⇨ 递归体。

⟷ 假设$n=k-1$时等式成立；求证$n=k$时等式成立

（3）确定一个特定情况（如$f(1)$或$f(0)$）的解 ⇨ 递归出口。

⟷ 求证$n=1$时等式成立

【例6.5】采用递归算法求整数数组a[0..n-1]中的最小值。

假设f(a，i)求数组元素a[0..i]（共i+1个元素）中的最小值。

- 当i=0时，有f(a，i)=a[0]。
- 假设f(a，i-1)已求出，显然有f(a，i)=MIN(f(a，i-1)，a[i])，其中MIN()为求两个值较小值函数。

得到递归模型

$$f(a,i)=\begin{cases} a[0] & \text{当 } i=0 \text{ 时} \\ \mathrm{MIN}(f(a,i-1),a[i]) & \text{其他情况} \end{cases}$$

$f(a, i)=a[0]$ 当$i=0$时

$f(a, i)=MIN(f(a, i-1), a[i])$ 其他情况

```
int Min(int a[],int i)                    //求a[0..i]中的最小值
{  if (i==0)                         //递归出口
      return a[0];
   else                              //递归体
   {  int mind=Min(a,i-1);           //递归调用
      return min(mind,a[i]);         //合并
   }
}
```

6.2.2 基于递归数据结构的递归算法设计

递归数据结构的数据特别适合递归处理 ⇨递归算法

种瓜得瓜：递归性

数据：D={瓜的集合}

运算：Op={种瓜}

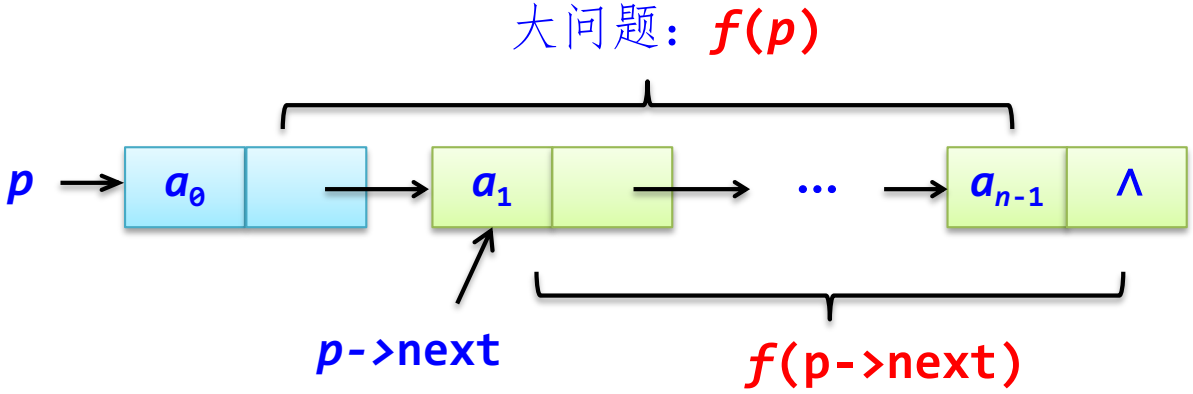
递归性：

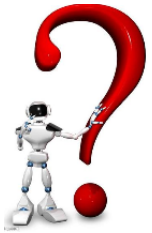
$Op(x \in D) \in D$

【例6.6】假设有一个不带头结点的单链表p，完成以下两个算法设计：

（1）设计一个算法正向输出所有结点值。

（2）设计一个算法反向输出所有结点值。



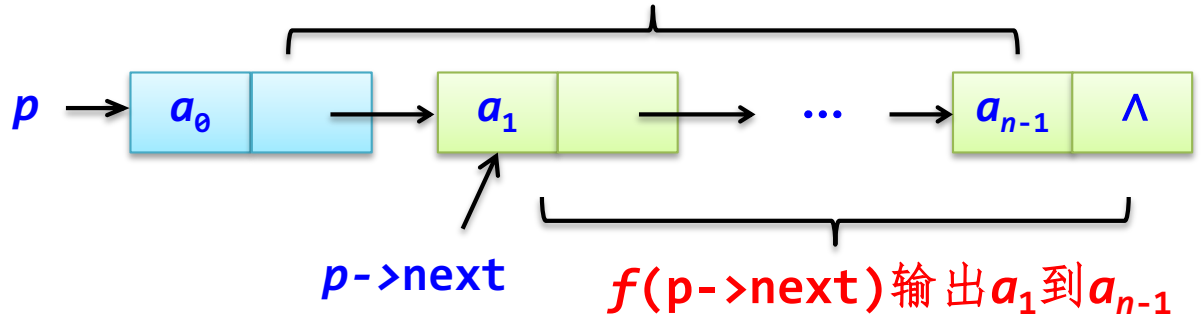


为什么在这里设计单链表的递归算法时不带头结点?

如何将带头结点转换为不带头结点的单链表?

(1)正向输出

大问题：$f(p)$输出a_0到a_{n-1}



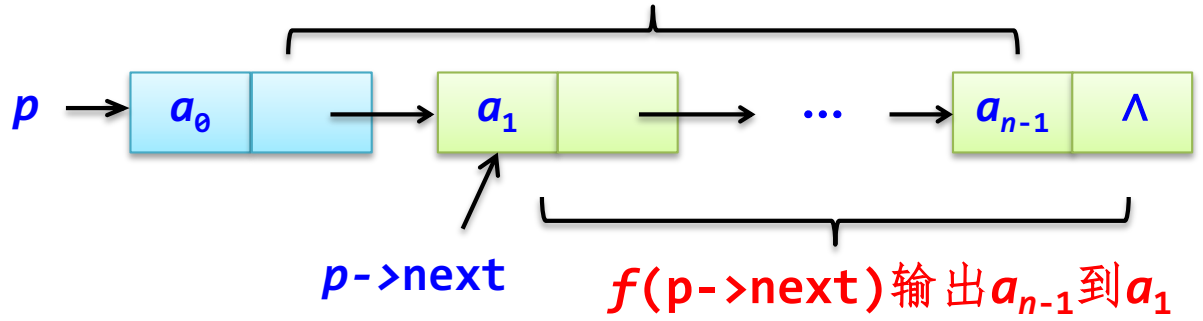
$f(p)$ ≡ 不做任何事件　　当p=NULL时

$f(p)$ ≡ 输出p结点值; f(p->next)　　其他情况

```
void Positive(LinkNode<int>* p)        //正向输出所有结点值
{  if (p==NULL)
     return;
   else
   {  printf("%d ",p->data);
      Positive(p->next);
   }
}
```

(2)反向输出

大问题：$f(p)$输出a_{n-1}到a_0



$f(p)$ ≡ 不做任何事件　　当p=NULL时

$f(p)$ ≡ f(p->next);输出p结点值　　其他情况

```
void Invert(LinkNode<int>* p)          //反向输出所有结点值
{  if (p==NULL)
      return;
   else
   {  Invert(p->next);
      printf("%d ",p->data);
   }
}
```

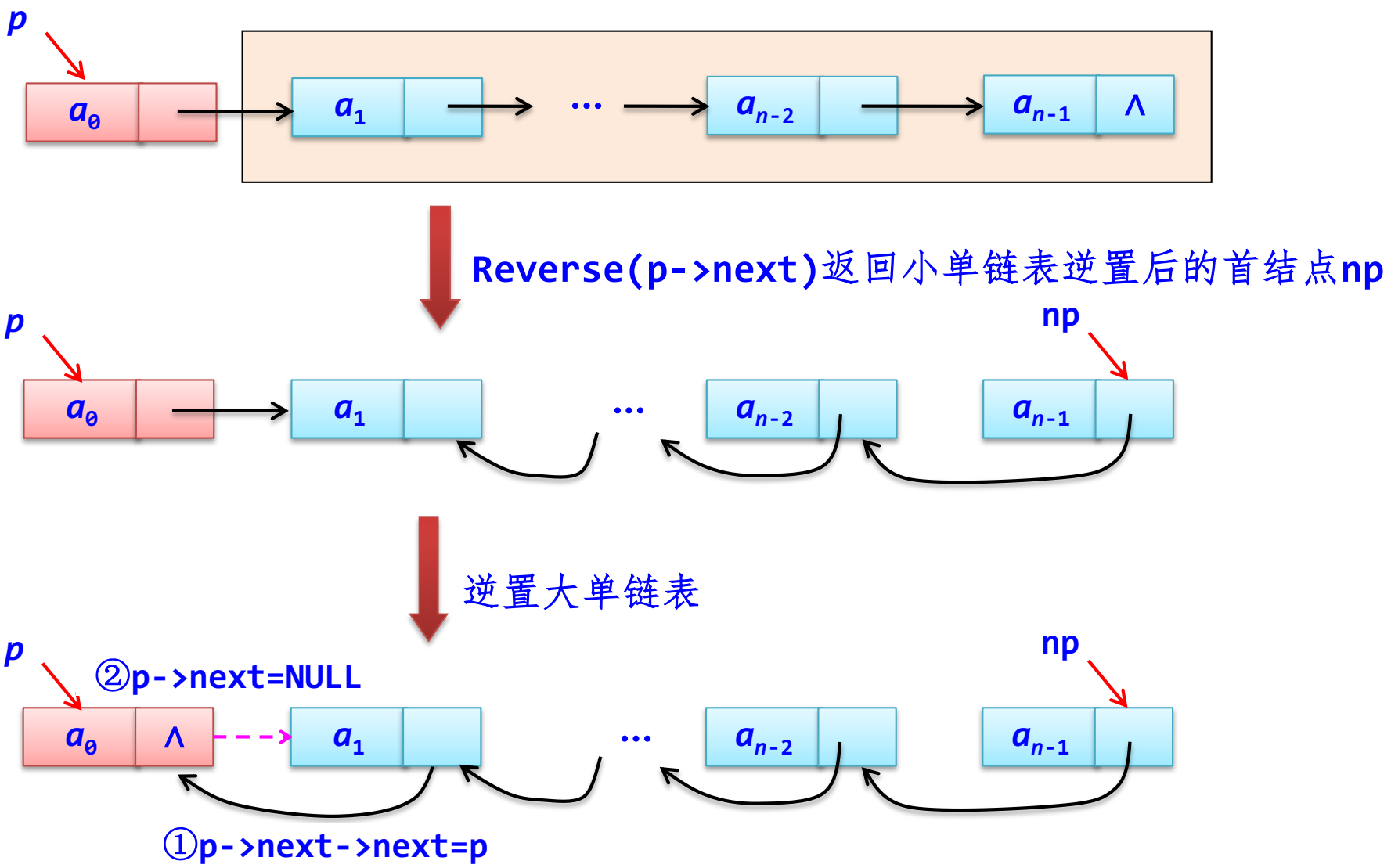
比较

```
void Positive(LinkNode<int>* p)        //正向输出所有结点值
{  if (p==NULL)
     return;
   else
   {  printf("%d ",p->data);
      Positive(p->next);
   }
}
```

```
void Invert(LinkNode<int>* p)          //反向输出所有结点值
{  if (p==NULL)
      return;
   else
   {  Invert(p->next);
      printf("%d ",p->data);
   }
}
```

【例6.7】假设有一个不带头结点的单链表*p*，设计一个递归算法逆置该单链表，例如，p->1->2->3->4逆置后为p->4->3->2->1。

- Reverse(p)的功能是逆置单链表*p*并返回逆置后单链表的首结点，为大问题。
- Reverse(p->next)为小问题。


p
a_0
a_1
…
a_{n-2}
a_{n-1}
Λ
Reverse(p->next)返回小单链表逆置后的首结点np
np
逆置大单链表
②p->next=NULL
①p->next->next=p

```
LinkNode<int>* Reverse(LinkNode<int>* p)     //逆置单链表p
{  if (p==NULL)                              //空表的情况
      return NULL;
   if (p->next==NULL)                        //只有一个结点的情况
      return p;
   else                                      //有2个及以上结点的情况
   {  LinkNode<int>* np;
      np=Reverse(p->next);                   //求解子问题
      p->next->next=p;                       //将结点p作为尾结点
      p->next=NULL;
      return np;                             //返回逆置单链表的首结点
   }
}
```

6.2.3 基于归纳方法的递归算法设计

- 通过对求解问题的分析归纳来转换成递归方法求解（如皇后问题等）。
- 关键是对问题本身进行分析，确定大、小问题解之间的关系，构造合理的递归体，而其中最重要的又是假设出“合理”的小问题。

【例6.8】若算法pow(*x*，*n*)用于计算x^n（*n*为大于1的整数）。完成以下任务：

（1）采用递归方法设计pow(*x*，*n*)算法。

（2）问执行pow(*x*，10)发生几次递归调用？求pow(*x*,*n*)对应的算法复杂度是多少？

（1）设$f(x, n)$用于计算x^n，则有以下递归模型：

$f(x, n)=x$	当$n=1$
$f(x, n)=x*f(x, n/2)*f(x, n/2)$	当n为奇数
$f(x, n)=f(x, n/2)*f(x, n/2)$	当n为偶数

```
double pow(double x,int n)    //求x的n次幂
{  if (n==1) return x;
   double p=pow(x,n/2);
   if (n%2==1)                //n为奇数
      return x*p*p;
   else                       //n为偶数
      return p*p;
}
```

（2）执行pow(*x*，10)的递归调用顺序是：

pow(*x*，10) → pow(*x*，5) → pow(*x*，2) → pow(*x*，1)

共发生4次递归调用。

求pow(*x*，*n*)对应的算法复杂度是$O(\log_2 n)$。

【例6.9】创建一个*n*阶螺旋矩阵并输出。例如，*n*=4时的螺旋矩阵如下：

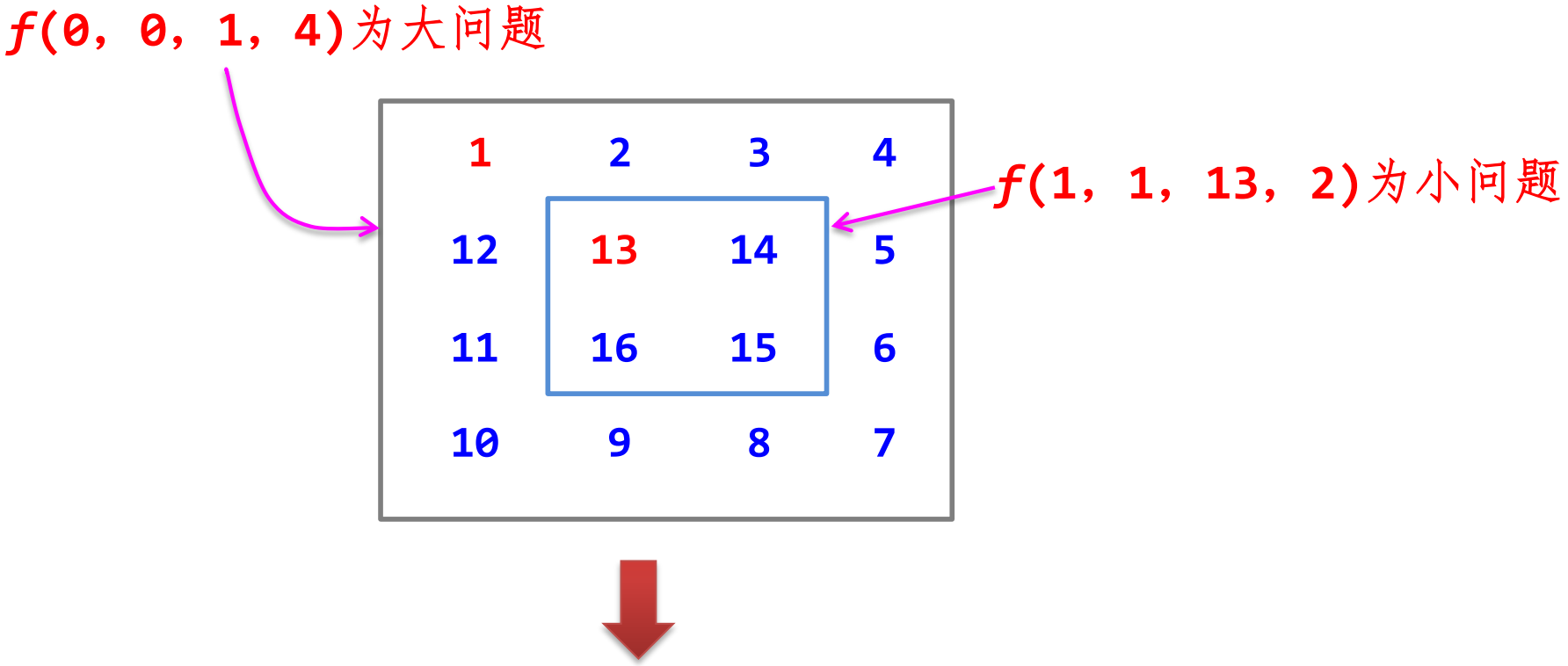
```
1    2    3    4
12   13   14   5
11   16   15   6
10   9    8    7
```

- 设*f*(*x*，*y*，start，*n*)用于创建左上角为（*x*，*y*）、起始元素值为start的*n*阶螺旋矩阵，共*n*行*n*列，它是大问题。
- *f*(*x*+1，*y*+1，start，*n*-2)用于创建左上角为（*x*+1，*y*+1）、起始元素值为start的*n*-2阶螺旋矩阵，共*n*-2行*n*-2列，它是小问题。

f(0，0，1，4)为大问题

1	2	3	4
12	13	14	5
11	16	15	6
10	9	8	7

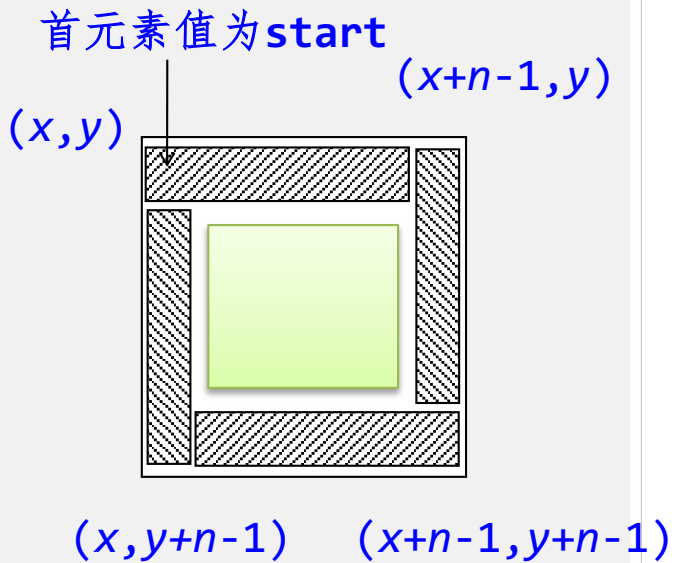
f(1，1，13，2)为小问题



对应的递归模型如下：

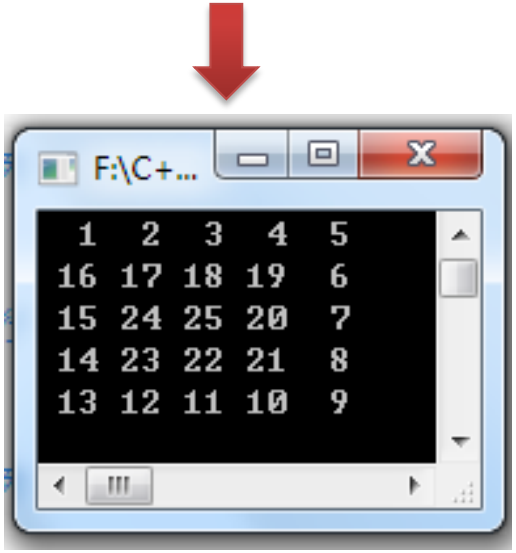
$f(x, y, start, n)$ ≡ 不做任何事情	当$n \leqslant 0$
$f(x, y, start, n)$ ≡ 产生只有一个元素的螺旋矩阵	当$n=1$
$f(x, y, start, n)$ ≡ 产生（x，y）的那一圈； $f(x+1, y+1, start, n-2)$	当$n>1$

```
void Spiral(int x,int y,int start,int n)    //递归创建螺旋矩阵
{  if (n<=0) return;                        //递归结束条件
   if (n==1)                                //矩阵大小为1时
   {  a[x][y]=start;
      return;
   }
   for (int j=x;j<x+n-1;j++) //上一行
   {  a[y][j]=start;
      start++;
   }
   for (int i=y;i<y+n-1;i++) //右一列
   {  a[i][x+n-1]=start;
      start++;
   }
   for (int j=x+n-1;j>x;j--) //下一行
   {  a[y+n-1][j]=start;
      start+=1;
   }
   for (int i=y+n-1;i>y;i--) //左一列
   {  a[i][x]=start;
      start++;
   }
   Spiral(x+1,y+1,start,n-2);//递归调用
}
```



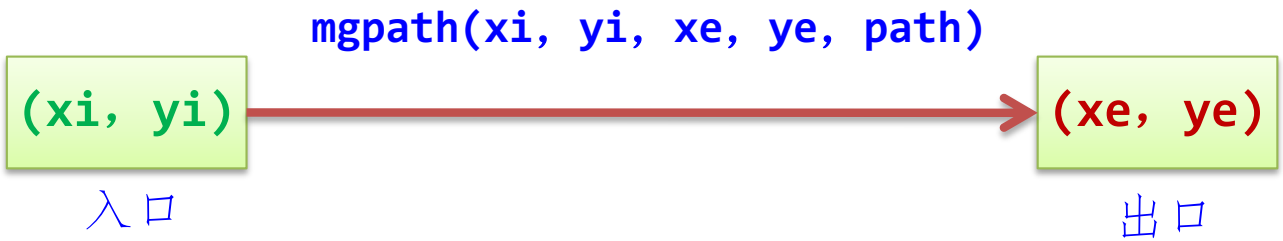
程序验证

```
int main()
{   int n=5;
    Spiral(0,0,1,n);
    for (int i=0;i<n;i++)
    {   for (int j=0;j<n;j++)
          printf("%3d",a[i][j]);
        printf("\n");
    }
}
```

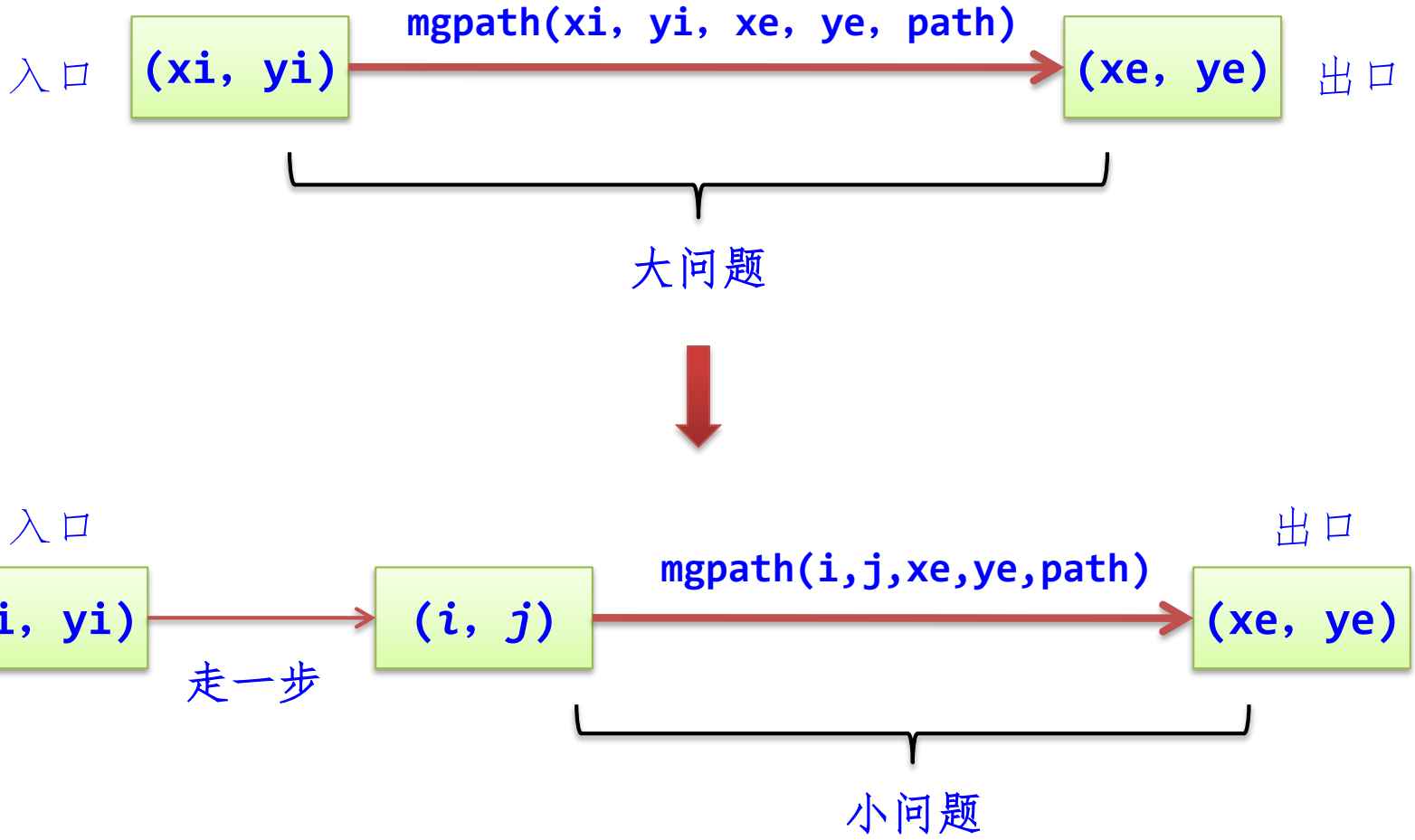


【例6.10】采用递归算法求解迷宫问题，并输出从入口到出口的所有迷宫路径。

求解问题描述：



mgpath(int xi，int yi，int xe，int ye，Box path)：求从(xi，yi)到(xe，ye)的迷宫路径，用path变量保存迷宫路径。


入口
(xi, yi)
mgpath(xi, yi, xe, ye, path)
(xe, ye)
出口
大问题
入口
(xi, yi)
走一步
(i, j)
mgpath(i,j,xe,ye,path)
出口
(xe, ye)
小问题

大问题 ≡ 走一步 + 小问题

求解迷宫问题的递归模型如下：

```
mgpath(xi,yi,xe,ye,path) ≡ 将(xi,yi)添加到path中;
                           置mg[xi][yi]=-1;
                           输出path中的迷宫路径;
                           恢复出口迷宫值为0即置mg[xe][ye]=0
                                    若(xi,yi)=(xe,ye)即找到出口
mgpath(xi,yi,xe,ye,path) ≡ 将(xi,yi)添加到path中;
                           置mg[xi][yi]=-1;
                           对于(xi,yi)每个相邻可走方块(i,j),
                                调用mgpath(i,j,xe,ye,path);
                           从(xi,yi)回退一步即置mg[xi][yi]=0;
                                    若(xi,yi)不是出口
```

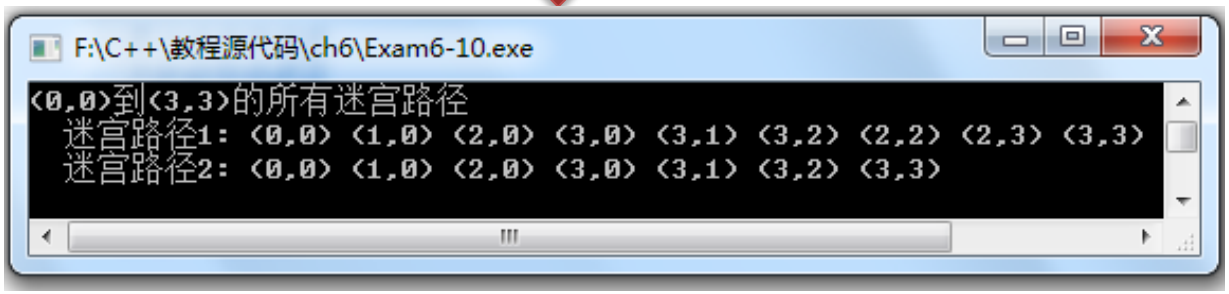
```
#include<iostream>
#include<vector>
using namespace std;
const int MAX=10;                                   //迷宫最大的行、列数
int dx[]={-1,0,1,0};                                //x方向的偏移量
int dy[]={0,1,0,-1};                                //y方向的偏移量
int mg[MAX][MAX]={{0,1,0,0},{0,0,1,1},{0,1,0,0},{0,0,0,0}};
int m=4,n=4;
int cnt=0;                                          //累计迷宫路径数
class Box                                           //方块类
{
public:
   int i;                                           //方块的行号
   int j;                                           //方块的列号
   Box(int i1,int j1):i(i1),j(j1) {}                //重载构造函数
};
```

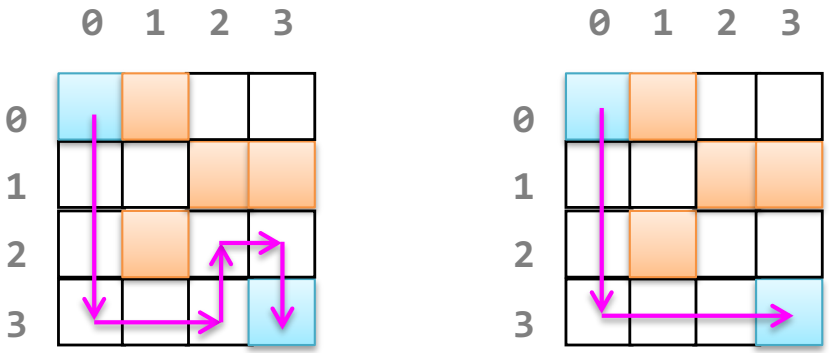
```
void mgpath(int xi,int yi,int xe,int ye,vector<Box> path)
//求解迷宫路径为:(xi,yi)->(xe,ye)
{  Box b(xi,yi);                          //建立入口方块的对象b
   path.push_back(b);                     //将b添加的路径path中
   mg[xi][yi]=-1;                         //mg[xi][yi]=-1
   if (xi==xe && yi==ye)                  //找到了出口,输出一个迷宫路径
   {  cnt++;
      printf("  迷宫路径%d: ",cnt);   //输出第cnt条迷宫路径
      for (int k=0;k<path.size();k++)
          printf("(%d,%d) ",path[k].i,path[k].j);
      printf("\n");
      mg[xi][yi]=0;                       //从出口回退，恢复其mg值
      return;
   }
```

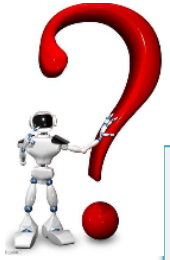
```
  else                                          //(xi,yi)不是出口
  { int di=0;
    while (di<4)                                //处理(xi,yi)四周每个相邻方块(i,j)
    { int i=xi+dx[di];                          //找(xi,yi)的di方位的相邻方块(i,j)
      int j=yi+dy[di];
      if (i>=0 && i<m && j>=0 && j<n && mg[i][j]==0)
         mgpath(i,j,xe,ye,path);                //若(i,j)可走,从(i,j)查找路径
      di++;                                     //继续处理(xi,yi)的下一个相邻方块
    }
    mg[xi][yi]=0;                               //(xi,yi)所有相邻方块处理完,回退
  }
}
```

程序验证

```
int main()
{   int xi=0,yi=0,xe=3,ye=3;
    printf("(%d,%d)到(%d,%d)的所有迷宫路径\n",xi,yi,xe,ye);
    vector<Box> path;
    mgpath(xi,yi,xe,ye,path);
    return 0;
}
```







迷宫问题的递归求解与用栈和队列求解有什么异同？

递归函数设计中几个问题

❶ 递归函数中的引用形参可以用全局变量代替

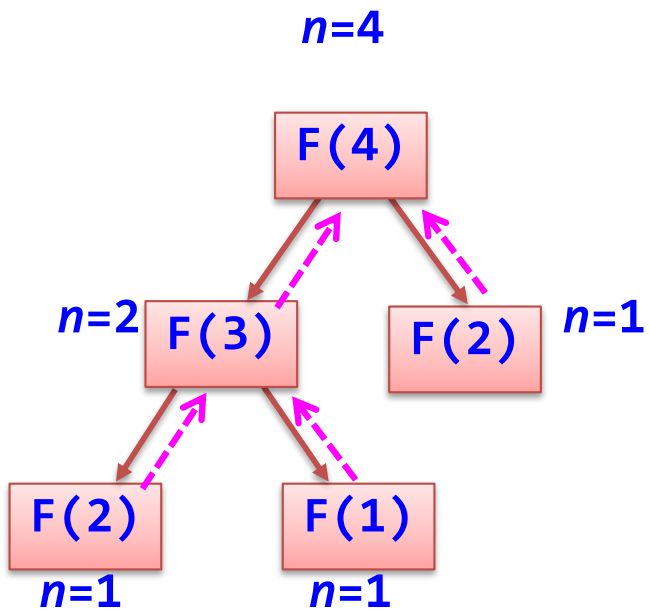
例如，求1 + 2 + … + *n*

```
void add(int n, int &s)     //s=1+2+…+n
{   int s1;
    if (n==1)
        s=1;
    else
    {    add(n-1, s1);
         s=s1+n;
    }
}
```

可以用全局变量代替：

```
int s=0;                    //全局变量
void add1(int n)            //理解为:s与add(n)绑定,  s=1+2+…+n
{  if (n==1)
        s=1;
   else
   {    add1(n-1);
        s+=n;
   }
}
```

❷ 递归函数中的非引用形参作为状态变量，可以自动回溯



- n表示状态
- 状态自动回溯

6.3 递归算法转换为非递归算法

自学

6.3 递归算法转换为非递归算法

6.3.1 迭代转换法

- 尾递归和单向递归是两种特殊类型的递归，可以采用迭代转换法将它们转换为非递归算法，即将其递归结构用循环结构来替代。
- 尾递归是递归调用语句只有一个，而且是处于算法的最后。
- 单向递归是指执行过程总是朝着一个方向进行的，递归函数中虽然有一处以上的递归调用语句，但各次递归调用语句的参数只和主调用函数有关，参数相互之间无关。

【例6.11】采用非递归算法求Fibonacci数列。

```
int Fib(int n)                          //求Fibonacci数列的第n项
{  if (n==1 || n==2)
      return 1;
   else
      return Fib(n-1)+Fib(n-2);
}
```

```
int Fib1(int n)                         //求Fibonacci数列的非递归算法
{  if (n==1 || n==2)
     return 1;
   int a=1,b=1,c;
   for (int i=3;i<=n;i++)
   {  c=a+b;
      a=b;
      b=c;
   }
   return c;
}
```

6.3.2 用栈模拟转换法

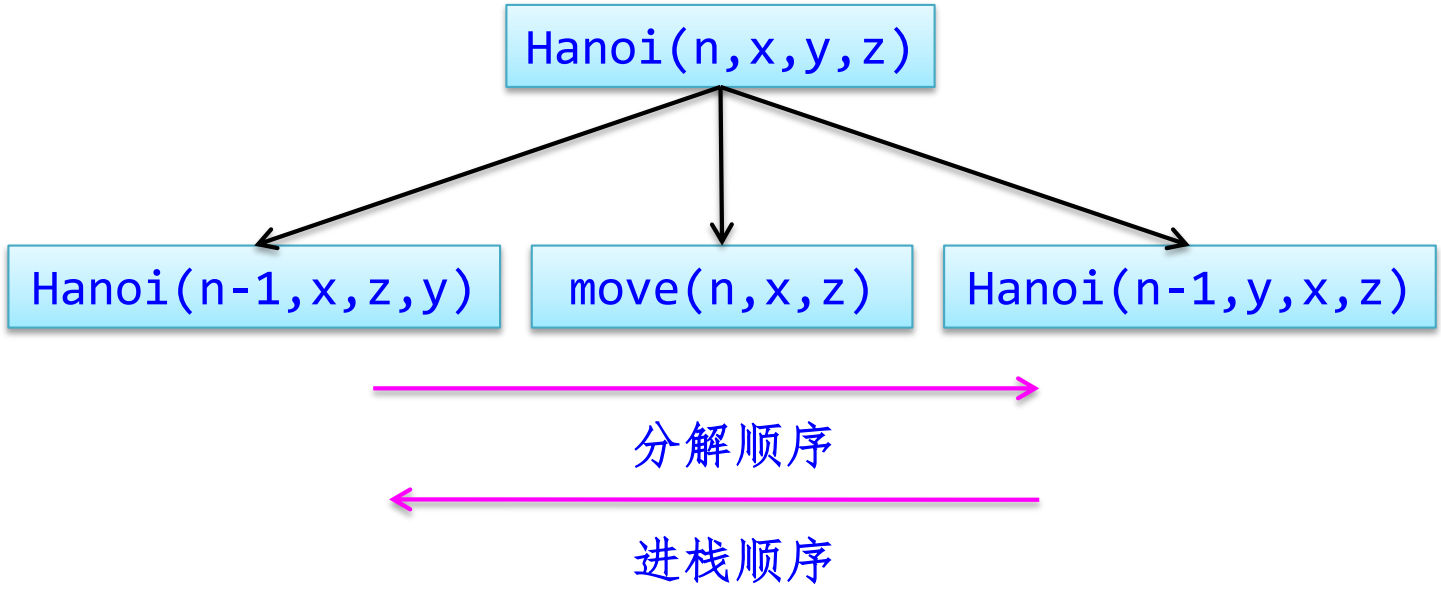
- 递归算法是一种分而治之的方法，把“大问题”求解转换为若干个相似“子问题”求解。
- 不能采用迭代转换法的递归算法执行中往往涉及回溯，可以采用栈来保存暂时不能执行的子问题，或者使用栈保存中间结果，称为用栈模拟转换法。
- 用栈模拟转换法的一般框架如下：

```
void nonrecursive(si) //非递归算法框架
{   将大问题状态si进栈;
    while (栈不为空)
    {   退栈一个元素s;
        if (s可以直接解决)
            直接求该问题解;
        else
        {   根据递归过程将s转换为若干个相似子问题的状态sj;
            将每个sj进栈;
        }
    }
}
```

【例6.12】采用非递归算法求Hanoi问题。

```
void Hanoi(int n,char x,char y,char z)      //Hanoi递归算法
{  if (n==1)                                //只有一个盘片的情况
      printf("将第%d个盘片从%c移动到%c\n",n,x,z);
   else                                     //有两个或多个盘片的情况
   {  Hanoi(n-1,x,z,y);
      printf("将第%d个盘片从%c移动到%c\n",n,x,z);
      Hanoi(n-1,y,x,z);
   }
}
```

- 当*n*>1时，需要将Hanoi(n，x，y，z)转换成Hanoi(n-1，x，z，y)、move(n，x，z)、Hanoi(n-1，y，x，z)三步。
- 用一个栈暂时存放Hanoi(n-1，x，z，y)和Hanoi(n-1，y，x，z)两步。
- 栈的特点是先进后出，所以要先将Hanoi(n-1，y，x，z)进栈，后将Hanoi(n-1，x，z，y)进栈。

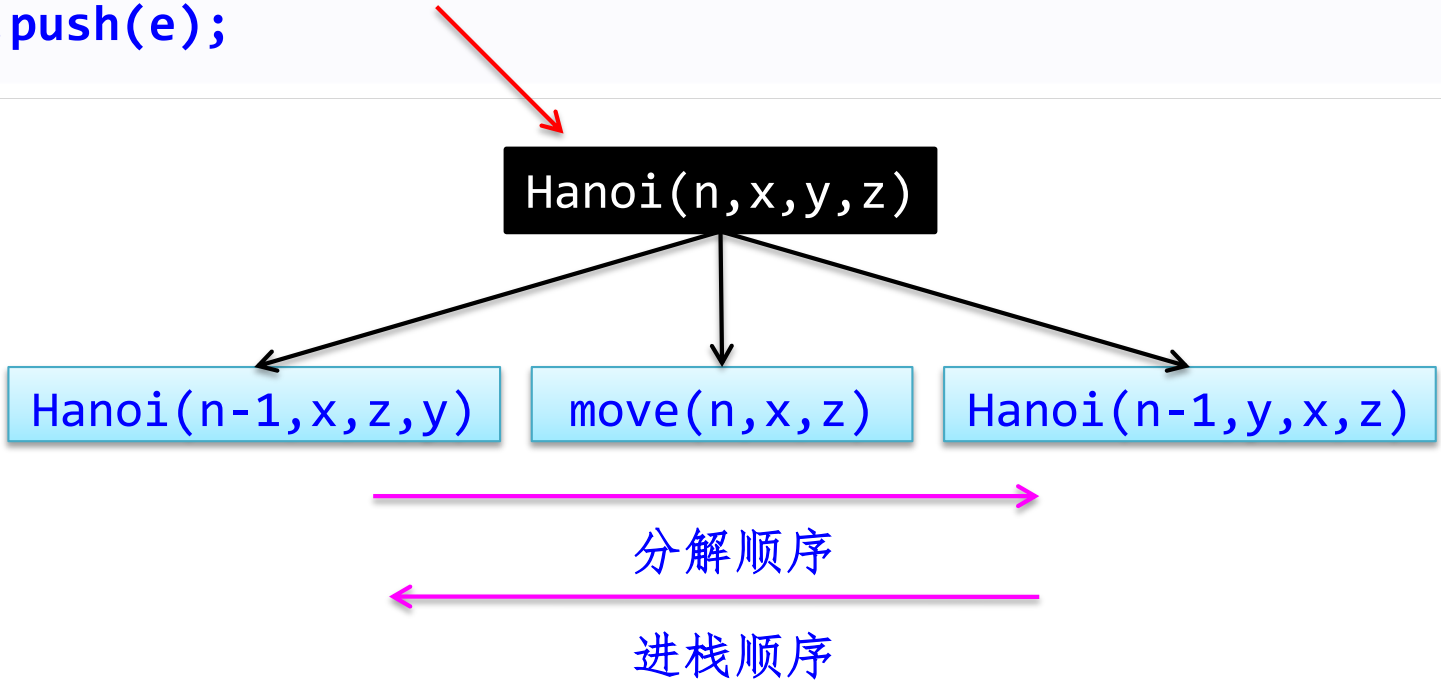


这里栈st采用顺序栈，其元素类型定义如下：

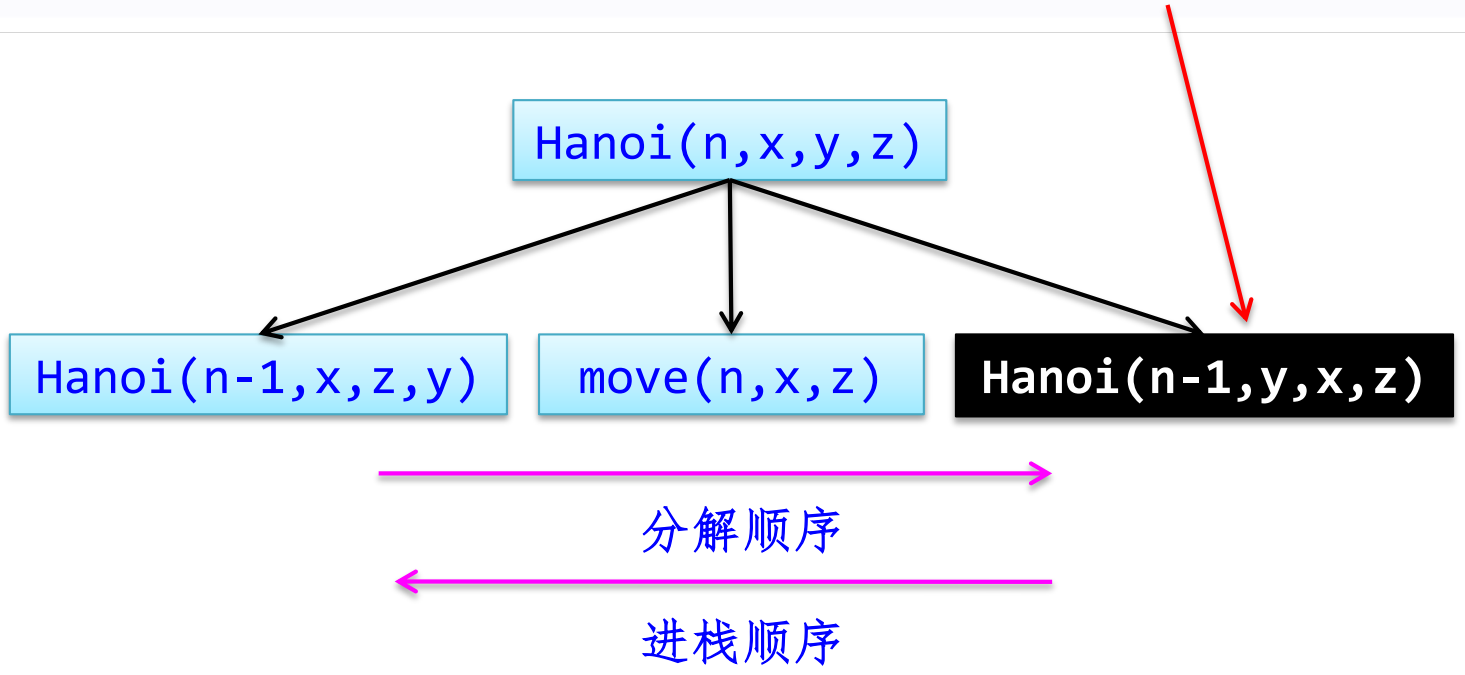
```
struct SNode                                          //栈元素类型
{  int n;
   char x,y,z;
   bool flag;                                         //是否可以直接移动

   SNode() {}                                                 //构造函数
   SNode(int n1,char x1,char y1,char z1,bool f1)    //重载构造函数
   {  n=n1;
      x=x1; y=y1; z=z1;
      flag=f1;
   }
};
```

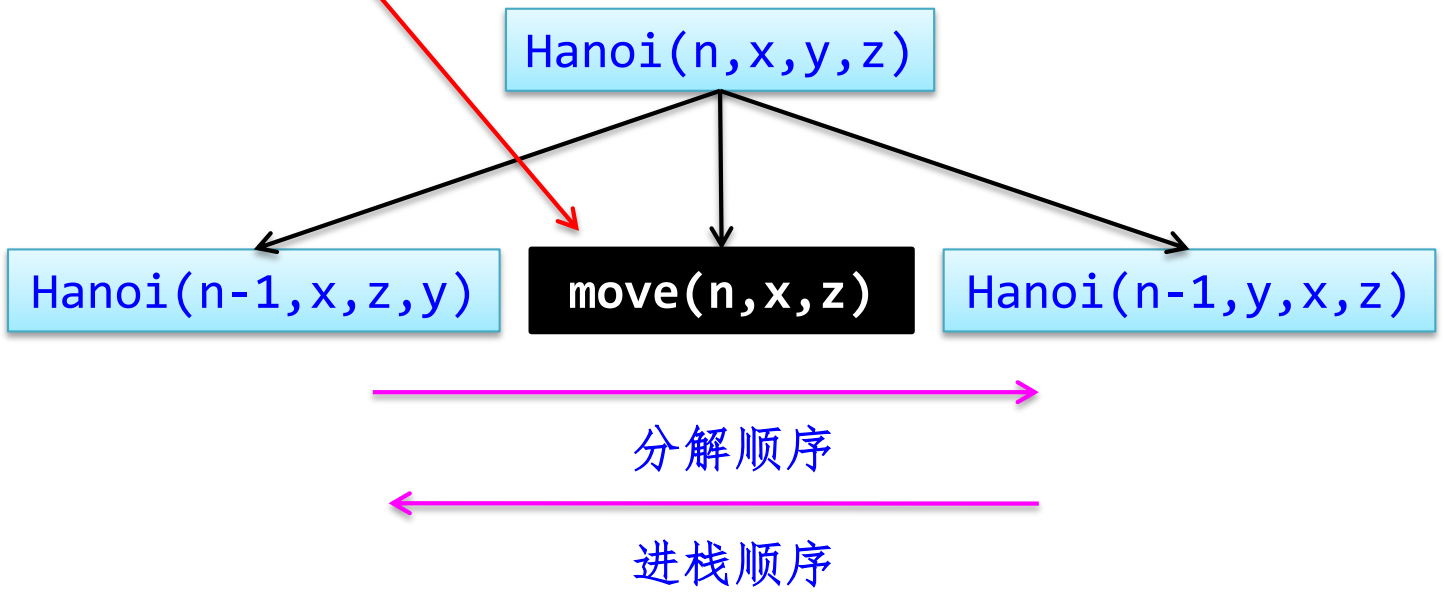
```
void Hanoi1(int n,char x,char y,char z)
//Hanoi问题的非递归算法
{  if (n==1)                          //只有一个盘片时直接移动
   {  cout << "盘片" << n << "从" << x << "移动到" << z << endl;
      return;
   }
   SNode e,e1,e2,e3;
   stack<SNode> st;                   //定义一个栈st
   e=SNode(n,x,y,z,false);
   st.push(e);
```



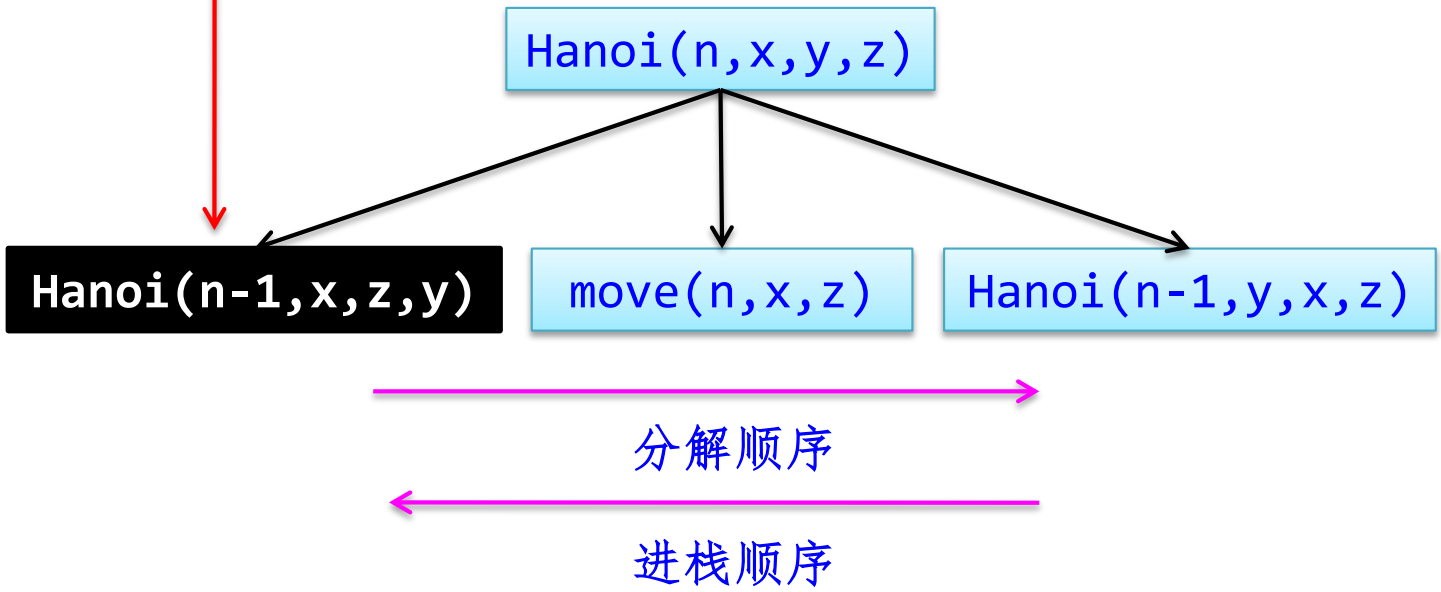
```
while (!st.empty())              //栈不空时循环
{  e=st.top(); st.pop();         //出栈元素e,对应任务为Hanoi(n1,x1,y1,z1)
   bool flag1=e.flag;
   char x1=e.x,y1=e.y,z1=e.z;
   int n1=e.n;
   if (flag1)                    //该任务可以直接移动
      cout << "盘片" << n1 << "从" << x1 << "移动到" << z1 << endl;
   else
   {  if (n1-1==1)
         e1=SNode(n1-1,y1,x1,z1,true);
      else
         e1=SNode(n1-1,y1,x1,z1,false);
      st.push(e1);               //Hanoi(n1-1,y1,x1,z1)任务进栈
```



```
e2=SNode(n1,x1,' ',z1,true);
st.push(e2);            //move(n1,x1,z1)任务进栈
```



```
            if (n1-1==1)
               e3=SNode(n1-1,x1,z1,y1,true);
            else
               e3=SNode(n1-1,x1,z1,y1,false);
            st.push(e3);              //Hanoi(n1-1,x1,z1,y1)任务进栈
        }
    }
}
```

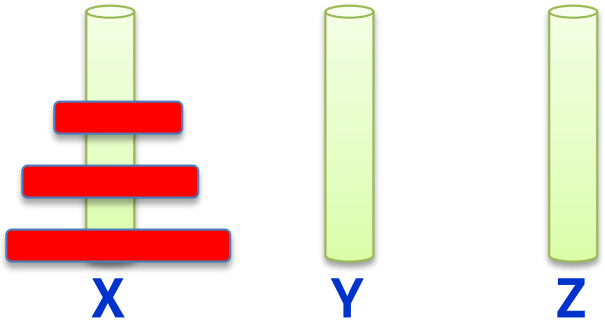


Hanoi(n, x, y, z), n=3 执行过程：

❶ Hanoi(n-1, X, Z, Y)任务

❷ Move(n, X, Z): flag=true

❸ Hanoi(n-1, Y, X, Z)任务



1	X	Y	Z	true
2	X	Z	Y	true
1	Y	Z	X	true
2	Y	X	Z	true
1	X	Y	Z	true

屏幕显示：

将第1个盘片从X移到Z

将第2个盘片从X移到Y

将第1个盘片从Z移到Y

将第3个盘片从X移到Z

将第1个盘片从Y移到X

将第2个盘片从Y移到Z

将第1个盘片从X移到Z

1.打印所有有效的括号组合

给定一个数 n，打印所有长度为 2n 的有效括号组合。

注意：如果每一个左括号 (都有对应的右括号) 且顺序正确，那么这个括号序列是有效的。

示例

输入1： n=1

输出1： ()

输入2： n=2

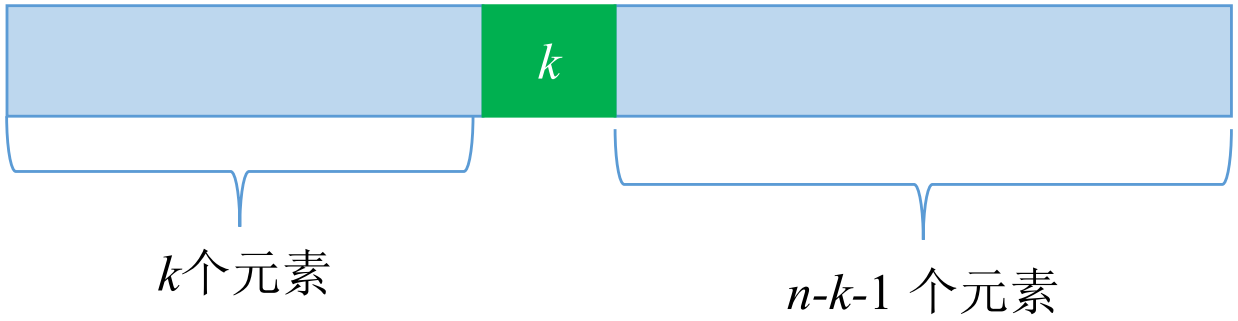
输出2： (()), ()()

输入3： n=3

输出2： (()), ()()

Catalan数

给定一个入栈序列，序列长度为**n**，请问有多少种出栈序列？



$$f(N)=\sum_{k=0}^{n-1} f(k)\times f(N-1-k)$$

计算方法

(1) $C_n = C_0C_{n-1} + C_1C_{n-2} + \ldots + C_{n-2}C_1 + C_{n-1}C_0 = \sum C_kC_{n-k}, \quad C_0 = 1$

(2) $C_n = \dfrac{4n-2}{n+1}C_{n-1}, \quad C_0 = 1$

(3) $C_n = \dfrac{1}{n+1}\binom{2n}{n} = \dfrac{(2n)!}{(n+1)!n!}$

部分Catalan数：1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, 208012, 742900, 2674440, 9694845, 35357670 ......

递归法（卡特兰数结构）

有效括号序列可分解为：'(' + A + ')' + B

其中 A 和 B 分别是有效括号序列，且 |A| + |B| = 2(n-1)。

遍历所有可能的内部括号对数 i（A 包含 i 对括号），则 B 包含 n-1-i 对括号。

n=0, P_0 =1

输入1: n=1, $P_1 = P_0*P_0$ =1

输出1: ()

输入2: n=2, $P_2 = P_1*P_0 + P_0*P_1$ = 1*1 + 1*1 =2

输出2: (()), ()()

输入3: n=3, $P_3 = P_2*P_0 + P_1*P_1 + P_0*P_2$ =1*2+1*1+2*1= 5

输出3: ((())), (()()), (())(), ()(()), ()()()

```
vector<string> allParenthesis(int n) {
        if (n == 0) return {""}; // 空序列
        vector<string> result;
        // i 表示内部括号对数，范围 [0, n-1]
        for (int i = 0; i < n; ++i) {
            vector<string> inside = allParenthesis(i); // A
            vector<string> outside = allParenthesis(n - 1 - i); // B
            for (const string& in : inside) {
                for (const string& out : outside) {
                    result.push_back("(" + in + ")" + out);
                }
            }
        }
        return result;
}
```

时间复杂度：卡特兰数 $O(\frac{1}{n+1}\binom{2n}{n})$

空间复杂度：O(n)

回溯法（DFS+剪枝）

把构造过程看作一棵 **决策树**：

- 根节点为空字符串
- 每一步可以添加 '(' 或 ')'，但必须满足有效性约束
- 当字符串长度达到 2n 时，得到一个完整序列。
- 回溯法（DFS + 剪枝）就是深度优先遍历这棵树，并在不满足约束的分支上提前返回（剪枝）

在递归过程中，我们维护三个信息：

- cur：当前已经构建的括号字符串
- open：已经使用的左括号个数
- close：已经使用的右括号个数
- n：总共需要的括号对数（全局固定）

递归出口：当 open == n 且 close == n 时，cur 长度为 2n，是一个有效序列，加入答案集。

剪枝条件

- open<n时，加 ‘(’
- close < open，加 ‘)’

这两个条件 保证了搜索过程中永远不会产生无效的中间状态，因此最终得到的每个长度为 2n 的字符串一定是有效的。

回溯法（DFS+剪枝）

n=2

初始状态：cur="", open=0, close=0

├─ 加 '(' (open=0<2) → cur="(", open=1, close=0

│ ├─ 加 '(' (open=1<2) → cur="((", open=2, close=0

│ │ ├─ 加 ')' (close=0<2) → cur="(()", open=2, close=1

│ │ │ └─ 加')'(close=1<2)→cur="(())",open=2,close=2有效序列 ✅

│ │ └─ 加 '(' (open=2不小于2) → 不可行，剪枝

│ └─ 加 ')' (close=0<1) → cur="()", open=1, close=1

│ ├─ 加 '(' (open=1<2) → cur="()(", open=2, close=1

│ │ └─ 加')'(close=1<2)→cur="()()",open=2,close=2有效序列 ✅

│ └─ 加 ')' (close=1不小于open=1) → 不可行，剪枝

└─ 加 ')' (close=0<0? 不满足 close<open) → 第一步就剪枝

```
void allParenthesis2 (vector<string>& result, string& current,
                      int open, int close, int n){
        // 递归出口：左右括号都用完了
        if (open == n && close ==n) {
            result.push_back(current);
            return;
        }
        // 添加左括号
        if (open < n) {
            current.push_back('(');
            allParenthesis2(result, current, open + 1, close, n);
            current.pop_back();  // 撤销选择(回溯)
        }
        // 添加右括号
        if (close < open) {
            current.push_back(')');
            allParenthesis2(result, current, open, close + 1, n);
            current.pop_back();
        }
    }
};
```

时间复杂度：卡特兰数 $O(\frac{1}{n+1}\binom{2n}{n})$

空间复杂度：O(n)